

Towards a Pluggable and Implementation-Agnostic Learning-Based Congestion Control for QUIC

JASHANJOT SINGH SIDHU, Concordia University, Canada

ABDELHAK BENTALEB, Concordia University, Canada

QUIC is rapidly becoming the de facto transport standard for HTTP Adaptive Streaming (HAS), offering multiplexed, low-latency delivery over UDP. However, most existing QUIC congestion control (CC) algorithms are heuristic-based and largely inherited from TCP designs, limiting their adaptability across diverse network conditions and ultimately degrading the user's Quality of Experience (QoE). Learning-based CC approaches have shown promise in TCP but are typically non-modular and tightly coupled to specific implementations, making their integration with QUIC impractical. To address these limitations, we propose AERO—the first learning-based, plug-and-play congestion control algorithm designed for portable integration across QUIC stacks. AERO dynamically adapts to varying network characteristics and can be seamlessly integrated into any QUIC stack without extensive engineering effort. We conduct extensive trace-driven evaluations to assess AERO's performance from both the transport and application layers, examining its influence on client-driven adaptive bitrate (ABR) algorithms. Experimental results show that AERO enhances video quality (VMAF) by approximately 13% and reduces rebuffering by 30% in video-on-demand scenarios, while achieving around 12% VMAF improvement and 65% fewer stalls in low-latency live streaming compared to state-of-the-art approaches. Moreover, AERO performs favorably in terms of delivery rate and delay across diverse network conditions, demonstrating its robustness and QUIC-friendliness.

CCS Concepts: • **Networks** → **Transport protocols**; • **Information systems** → **Multimedia streaming**.

Additional Key Words and Phrases: CC, QUIC, ABR, DRL, LLL, QoE.

1 Introduction

The latest Global Internet Phenomena Report [1] highlights that video streaming dominates Internet traffic, accounting for approximately 65% of total traffic in 2023. This rapid growth is largely driven by major platforms such as YouTube, Netflix, and Meta, which serve vast global audiences. These industry leaders predominantly rely on HTTP Adaptive Streaming (HAS) [8] to deliver content efficiently. HAS enables video clients to dynamically adjust playback quality based on current network conditions, ensuring a smoother and more uninterrupted viewing experience for users.

For many years, Transmission Control Protocol (TCP) has been the dominant transport protocol for HAS-based traffic. The recent emergence of Google's QUIC [31] represents a significant shift in network communication and media delivery [17], with platforms such as YouTube and Meta increasingly serving video content over QUIC. QUIC introduces several advanced features—including multiplexing, 0-RTT connection establishment, request prioritization, and seamless connection migration—that make it a compelling alternative to TCP, particularly for HAS-based streaming. Built on top of UDP and designed to support HTTP/3, QUIC provides a more efficient and secure transport mechanism, enhancing viewer Quality of Experience (QoE) compared to conventional TCP-based solutions [31, 44, 47].

Authors' Contact Information: Jashanjot Singh Sidhu, Concordia University, Montreal, Canada; Abdelhak Bentaleb, Concordia University, Montreal, Canada.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 1551-6865/2026/7-ART

<https://doi.org/10.1145/3827616>

Typically, a server-side (i.e., sender) congestion control (CC) algorithm governs the data sending rate according to the network's capacity, whereas a client-side (i.e., receiver) adaptive bitrate (ABR) scheme selects the optimal bitrate and resolution for upcoming video segments based on factors such as available bandwidth and buffer occupancy. Although numerous QUIC implementations exist [16]—including Quiche, Mvfst, Quinn, and Aioquic—they all rely on heuristic CC algorithms originally designed for TCP. Common examples of these heuristics include Cubic [36], NewReno [36], BBR2 [36], and Copa [36]. Such traditional algorithms often struggle to adapt effectively to diverse network conditions [35, 36], motivating the adoption of learning-based CC approaches for TCP. By leveraging deep reinforcement learning (DRL) within the CC logic of TCP, prior works [2, 58] have demonstrated significant performance improvements. However, while the underlying neural network models of these approaches are in principle transport-agnostic, their integration mechanisms are tightly coupled to the Linux kernel's TCP stack (e.g., requiring modifications to `tcp_congestion_ops`). These kernel-level dependencies have no equivalent in QUIC, which implements congestion control entirely in user space [26]. Consequently, deploying these solutions in QUIC requires a fundamentally different integration approach, which AERO provides.

1.1 Problem and Motivation

To highlight the limitations of heuristic-based congestion control algorithms in QUIC, we conducted experiments using the Vegvisir framework [20]. Vegvisir comprises three components—server, traffic shaper, and client—each running in separate Docker containers and connected via Docker Compose networks to control traffic flow. At the server, we deployed Quiche, while at the client we used Quic-go [41] together with the goDash player [43] for HAS streaming. For bitrate adaptation, we employed Pensieve [37], a widely used learning-based ABR algorithm, and emulated network conditions using the LTE Belgium trace [54], which has an average bandwidth of 3.3 Mbps with a 1-second sampling interval to introduce realistic bandwidth fluctuations between client and server.

The traffic shaper (tc Netem) was employed to emulate diverse network conditions based on real-world traces. Each experiment was repeated three times, with a duration of 120 seconds per run, and the results were averaged. The ideal behavior for a CC algorithm is to maximize the utilization of available link capacity while avoiding excessive queuing and packet loss. In practice, exactly matching the instantaneous link capacity is not feasible due to inherent feedback delays, measurement noise, and the bursty nature of video segment requests. Instead, a well-performing CC algorithm should closely track the link capacity over time, responding promptly to capacity increases without overshooting significantly during decreases. As shown in Fig. 1, the CDF of delivery rates reveals that BBR2, Cubic, and Reno significantly under-utilize the available link capacity, with their distributions concentrated at low delivery rates.

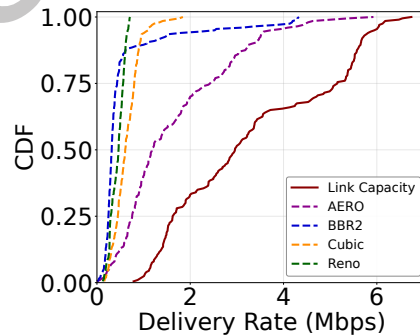


Fig. 1. CDF of delivery rate for different CC algorithms.

As shown in Fig. 1, the CDF of delivery rates reveals that BBR2, Cubic, and Reno significantly under-utilize the available link capacity, with their distributions concentrated at low delivery rates. These congestion control algorithms struggle to increase their delivery rate [10], with their congestion windows (cwnd) remaining far below optimal levels. For instance, BBR2 achieves a median delivery rate of only ~ 0.32 Mbps, while Cubic and Reno reach ~ 0.61 Mbps and ~ 0.46 Mbps, respectively. In contrast, our proposed solution AERO [46] (detailed in Section 3) dynamically adapts to the available link capacity, achieving a median delivery rate of ~ 1.18 Mbps, approximately $3.7\times$ that of BBR2 and $1.9\times$ that of Cubic. We note that AERO does not aim to perfectly match the instantaneous link capacity; rather, it achieves substantially higher utilization than heuristic-based alternatives by reacting more promptly to capacity fluctuations through its learned policy, demonstrating its efficiency and robustness across varying network conditions.

1.2 AERO: A DRL-based CC for QUIC

To overcome the limitations of conventional congestion control, we propose AERO [46], a deep reinforcement learning (DRL)-based CC solution that leverages network statistics—such as throughput, delay, jitter, and packet loss—to dynamically adjust the congestion window (cwnd) at each congestion event. This enables more effective adaptation to the available link capacity while supporting ABR schemes in making informed bitrate decisions, ultimately improving user QoE. By harnessing server-side computational resources and incorporating state-of-the-art DRL techniques, AERO enhances congestion control, leading to more efficient network utilization and robust performance in dynamic environments. Specifically, our approach employs an Advantage Actor-Critic (A2C) network architecture with Proximal Policy Optimization (PPO) to learn optimal policies, and Generalized Advantage Estimation (GAE) to improve learning by estimating the advantages of different actions over time. Network parameters are updated efficiently using the RMSProp optimizer, ensuring faster convergence during training. Importantly, AERO is designed for pluggability: it is a modular CC algorithm that can be easily integrated into any QUIC implementation and operates seamlessly with any ABR scheme, making it both implementation- and ABR-agnostic. By *implementation-agnostic*, we mean that AERO is not coupled to any specific QUIC stack—it imposes no assumptions about the underlying codebase, programming language, or internal architecture, requiring only access to four standard transport-layer signals and a writable congestion window variable. The same trained model can therefore be deployed across heterogeneous QUIC stacks without retraining or architectural changes, as demonstrated in Section 4.3.3. The technical basis for AERO’s pluggability, namely its minimal interface requirements and universal cwnd update primitive, is detailed in Section 3.3. To the best of our knowledge, AERO is the first pluggable, learning-based congestion control algorithm for QUIC, and this work represents the first study to systematically evaluate the impact of a QUIC CC on ABR performance. In this paper, we present four key contributions:

- (1) We propose AERO, a server-side, ABR-agnostic, learning-based, and pluggable congestion control algorithm for QUIC. AERO dynamically adjusts the congestion window (cwnd) based on current network conditions and assists any integrated ABR scheme in making more informed bitrate decisions, thereby improving both objective and perceptual QoE for end users.
- (2) We formulate the (cwnd) adjustment problem in QUIC as a Markov Decision Process (MDP) and employ an Advantage Actor-Critic (A2C) neural network architecture combined with Proximal Policy Optimization (PPO) and Generalized Advantage Estimation (GAE) to efficiently learn optimal policies and enable rapid adaptation to changing network conditions.
- (3) We implement AERO in Quiche and conduct extensive experiments across multiple dimensions, including diverse ABR schemes, network traces, congestion control algorithms, video content types and encoding parameters, as well as streaming modes such as video-on-demand and low-latency live. Experimental results demonstrate that AERO consistently outperforms competing approaches in terms of QoE, delivery rate, and latency.
- (4) We integrate AERO into multiple QUIC implementations—including Aioquic, Quinn, and Tquic—to evaluate its fairness and competitiveness across multiple clients. Experimental results demonstrate that AERO is QUIC-friendly, achieving both efficiency and fairness in multi-client scenarios. Importantly, AERO’s modular and implementation-agnostic design allows it to be seamlessly plugged into any QUIC stack, providing a blueprint for flexible, pluggable, learning-based congestion control for future QUIC deployments.

The remainder of the paper is organized as follows. Section 2 reviews existing congestion control and ABR solutions. Section 3 presents the design of AERO. Section 4 details the performance evaluation, followed by discussions in Section 5 and Section 6 concludes the paper.

2 Related Work

Server-driven heuristic-based CC. Heuristic-based congestion control (CC) algorithms are typically classified as loss-based, delay-based, or hybrid. Popular loss-based algorithms such as Cubic, NewReno, and Tahoe [36] adjust the congestion window (cwnd) primarily based on packet loss, making them poorly adaptive to variations in delay and throughput. Delay-based algorithms, including Vegas [36], Copa [36], and FAST [36], rely mainly on network delay for cwnd updates and thus struggle to handle packet loss and dynamic link capacities. Hybrid approaches, notably BBR [36] and BBR2 [36], consider both delay and loss; however, as demonstrated in Section 1.1, they often fail to fully utilize available link capacity.

Server-driven learning-based CC. PBQ [62] is currently the only known learning-based CC mechanism for QUIC, built atop BBR. Although it provides two levels of control, it still relies on a heuristic-based algorithm as the primary control layer. For TCP, several learning-based CC approaches exist. Remy [55] dynamically adjusts the sending rate based on network feedback, while Aurora [28] employs a simple fully connected neural network for congestion control. Orca [2] integrates an A2C network with reinforcement learning policies alongside Cubic, and Sage [58] uses an offline, off-policy DRL approach that does not require interaction with the environment. More recently, Astraea [34] introduces a multi-agent DRL framework that explicitly optimizes fairness, convergence speed, and stability among competing TCP flows. Jury [53] extends this direction by decoupling fairness enforcement from performance optimization through a preprocessing and post-processing pipeline, improving generalizability to unseen network conditions. Mutant [40] takes an online RL approach that learns in real time by dynamically switching between a pool of existing CC protocols in the Linux kernel, avoiding extensive offline training. AliCCS [61] proposes a learning-based CC selection framework for production CDNs that chooses the best-performing CC from a pool of existing algorithms based on fine-grained network domain knowledge. A common limitation of these TCP learning-based approaches is the need for kernel-level modifications, which complicates their integration with QUIC in user space. We note that the neural network models underlying these approaches are, in principle, transport-agnostic: a trained DRL policy is simply a function approximator. However, the distinction we draw is not about the model architecture but about the *integration mechanism*. These TCP-based solutions are implemented as kernel modules or require kernel-level patches (e.g., modifying `tcp_congestion_ops`) to intercept congestion events. QUIC implements congestion control entirely in user space [26] with implementation-specific APIs that vary across stacks, and therefore these kernel-level dependencies have no equivalent in QUIC. AERO addresses this gap by providing a DRL-based CC designed for user-space integration, as detailed in Section 3.3. While one could, in principle, extract the trained neural network weights from these TCP-based approaches and invoke them from user space within a QUIC stack, such models were trained end-to-end within TCP’s kernel-level feedback loop, where signal timing, congestion event semantics, and state update frequencies differ from those in QUIC. Reusing these models without retraining would yield misleading results, and properly adapting them to QUIC would require re-implementing the full training infrastructure within each QUIC stack’s user-space CC interface, a substantial engineering effort that underscores the need for a CC solution designed natively for QUIC, as AERO provides.

Modular QUIC designs. PQQUIC [12] proposes a pluginized QUIC framework that allows protocol extensions to be loaded dynamically at runtime via eBPF bytecode plugins. While PQQUIC provides a general plugin architecture, it has several limitations: (i) it is built on top of a single QUIC implementation (Picoquic) and is not portable across other QUIC stacks, (ii) plugins are constrained by the eBPF instruction set and sandboxed execution environment, which restricts computational capability and access to connection state, (iii) plugin exchange requires both endpoints to support the PQQUIC framework, precluding unilateral server-side deployment, and (iv) PQQUIC does not address learning-based congestion control. AERO is complementary in scope: it focuses specifically on providing a DRL-based CC module that can be integrated into any standard QUIC implementation without requiring a specialized plugin framework at both endpoints.

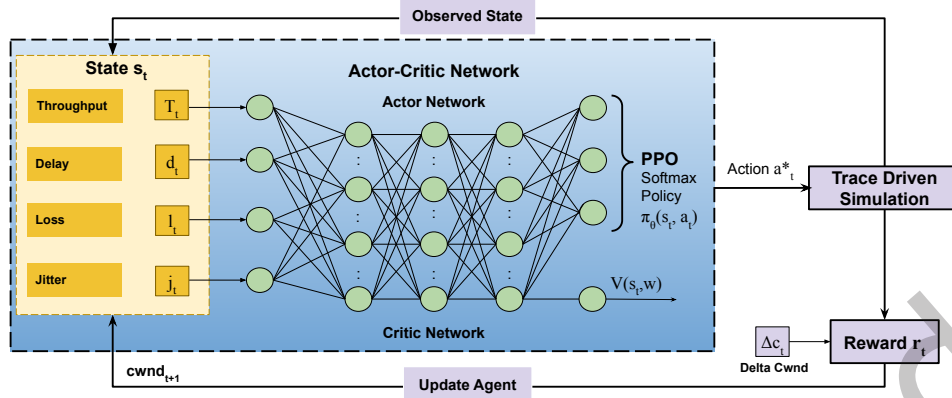


Fig. 2. AERO DRL architecture.

Client-driven HAS-based ABR. Heuristic-based ABR algorithms can be categorized as throughput-based (e.g., PANDA [33]), buffer-based (e.g., BOLA [49], BBA2 [23]), segment-size-based (e.g., SARA [5]), and hybrid (e.g., MPCDASH [59]). Learning-based ABRs include Pensieve [37], Merina [29], Fugu [56], Ahaggar [7], Comyco [22], and AMP [6], which leverage reinforcement learning or other machine learning techniques to optimize bitrate selection for improved QoE.

3 AERO Congestion Control for QUIC

In contrast to the conventional CC algorithms that use Additive Increase Multiplicative Decrease (AIMD) principle to update the congestion window (cwnd), AERO employs a deep reinforcement learning approach to dynamically adjust the cwnd during congestion events. While AIMD increases the cwnd linearly and decreases it multiplicatively, which can lead to slow convergence and suboptimal utilization under variable network conditions [57], AERO adapts more intelligently to network dynamics. Specifically, it leverages an actor-critic (A2C) architecture combined with Proximal Policy Optimization (PPO) for policy selection and updates, enabling faster and more precise adaptation. PPO's clipped objective function ensures stable training by limiting policy updates, allowing the network to learn more effectively and adapt to diverse environments. The overall A2C NN architecture of AERO is shown in Fig. 2.

3.1 Problem Formulation

At each time step t , AERO decides the optimal congestion window (denoted by cwnd_t^*) for the next state (denoted by s_{t+1}) based on the current state (denoted by s_t) by selecting a multiplicative factor (denoted by A^α). The cwnd update problem is mathematically described as follows:

$$\begin{cases} \text{find } \text{cwnd}_t^*(\pi), \forall t \in \{1, \dots, k\} \\ \arg \max_{\pi} r_t(\pi) \\ \text{s.t. } A \in [1.2, 3] \text{ C.1 and } \alpha \in [-1, 1] \text{ C.2,} \end{cases} \quad (1)$$

where cwnd_t^* denotes the optimal congestion window based on the current network statistics for s_{t+1} . π represents the RL policy responsible for determining the best cwnd update by adjusting both A and α . Here, k is the total number of steps, and r is the step reward (defined in (2)). The problem in (1) is formulated as a single-agent decision problem aimed at identifying the optimal cwnd update with respect to **C1** and **C2**, where the agent observes the environment state at each time step t . We fine-tuned the multiplicative factor A used to scale the congestion window and empirically constrained it to the range $A \in [1.2, 3]$. Values below 1.2 resulted in overly conservative growth behavior that closely resembled traditional AIMD-style window updates, leading to slow

bandwidth probing and under-utilization of the available link capacity. In contrast, values above 3 caused the window to expand too aggressively, which led to oscillatory cwnd dynamics, more frequent congestion events, and slower recovery. A detailed ablation analysis of the impact of selecting A outside this range is provided in Appendix D. The parameter α is naturally bounded by the $\tanh$ activation function. This setup follows the standard Markov Decision Process (MDP) formulation [15], where the agent selects actions based on its current state and observations to maximize long-term rewards. The MDP is defined as a 4-tuple $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R})$, where each element is described as:

- $\mathcal{S} = \{s_1, \dots, s_k\}$ is a set of finite and discrete agent states.
- $\mathcal{A} = \{a_1, \dots, a_k\}$ is a set of finite and continuous actions.
- $\mathcal{P} = \mathcal{S} \times \mathcal{S} \times \mathcal{A} \rightarrow [0, 1]$ is the state transition probability function $\mathcal{P}(s_{t+1}|s_t, a_t)$ from the state s_t to state s_{t+1} when action a_t is taken.
- $\mathcal{R} = \{r_1, \dots, r_k\}$ is the set of expected immediate rewards, which depends on the state and action taken by the agent at every time step t . $\mathcal{R}(s_t, a_t, s_{t+1}) (= r_t)$ is the step reward.

At each time step $t \in \{1, \dots, k\}$, the agent has access to the observations from the environment in the form of a state vector $\vec{s}_t$. It selects the right multiplicative factor for cwnd that maximizes the accumulated discounted advantage (denoted by Λ_t and defined in Section 3.2). The AERO model solves the MDP problem using a single agent A2C NN with clipped PPO, generalized advantage function (GAE), and RMSProp optimizer for policy (π^*).

3.2 AERO DRL Offline Training

We utilize the dataset provided by the Microsoft RL Bandwidth Estimation Challenge [39] to train the A2C NN of the AERO model. This dataset consists of a series of peer-to-peer Microsoft Teams audio/video calls conducted across a diverse set of testbed nodes spanning multiple countries and continents. These nodes were interconnected via the Internet, using a variety of Internet Service Providers (ISPs) and both wired and wireless connections.

Each entry in the trace is represented as a 150-dimensional observation vector derived from packet data received by the client during a single audio/video call. The vector records 15 metrics across fixed time monitor intervals (MI) of 60 ms. Additionally, the dataset provides the true bottleneck capacity and packet loss rate of the link. From the perspective of AERO, the selected metrics are summarized as follows:

- (1) *Receiving Rate* (bps): The rate at which the client receives data from the sender during a MI.
- (2) *Delay* (ms): The average delay of packets received during a MI minus a fixed base delay of 200 ms.
- (3) *Packet Jitter* (ms): The standard deviation of inter-arrival times of packets received in a MI.
- (4) *Packet Loss Ratio*: The probability of packet loss during a MI.
- (5) *Packets Probability*: The proportion of video, audio, and probing packets among all packets received in a MI.
- (6) *Bottleneck Capacity* (bps): The actual link capacity.

We acknowledge that the Microsoft Teams dataset reflects peer-to-peer conversational traffic, whereas our evaluation focuses on client-server HAS streaming. However, AERO operates purely at the transport layer and relies only on congestion feedback signals (rate, delay, jitter, and packet loss), which are common to both workloads, though we acknowledge that their distributions and temporal structure may differ across application types. Importantly, the Teams dataset spans a wide range of real-world network conditions across heterogeneous ISPs, access technologies, and geographic regions, providing a diverse training distribution that encourages generalizable congestion responses rather than application-specific behavior. The primary difference between the two workloads lies in the temporal structure of media segment requests, not in the underlying network dynamics. As shown in our evaluation in Section 4.3, AERO transfers effectively to HAS streaming without any additional tuning. Nonetheless, we acknowledge that further adaptation (e.g., lightweight fine-tuning or online

reinforcement learning) could better align AERO to workload-specific latency and buffering characteristics, and we identify this as a direction for future extension.

To enhance generalization across diverse network conditions, we introduced linear noise to the delay in the range of $[-2, 100]$ ms. Additionally, since most of the data had zero packet loss, we added linear noise to the loss metric in the range of $[0, 5]\%$, thereby augmenting the observation space for our agent. Originally, the dataset consisted of discrete audio/video calls, with observation vectors derived from distinct time intervals, resulting in a discrete state-action space for our MDP. This discrete representation proved sub-optimal when used with PPO. By adjusting the reward based on the bottleneck capacity, as detailed in Algorithm 1, we effectively transformed the discrete state-action space into a semi-continuous state-action space, improving learning efficiency and policy adaptability. The solution of the MDP is a policy that prescribes which action to take in each state. Formally, the RL agent interacts with an environment defined by a state space $\mathcal{S}$ and an action space $\mathcal{A}$. At each time step t , the agent observes a state $s_t \in \mathcal{S}$, takes an action $a_t \in \mathcal{A}$, and receives a reward $r_t \in \mathcal{R}$. The objective of the agent is to find the optimal policy $\pi^* : \mathcal{S} \rightarrow \mathcal{A}$ that maps states to actions while maximizing the cumulative reward.

▷ **Input State.** At each time step t , the agent receives the current state defined as $s_t = \{th_t, d_t, l_t, j_t\}$, comprising throughput (bps), delay (ms), packet loss ratio (%), and jitter (ms). The state s_t is fed into the A2C network as a 4-dimensional vector $\vec{s}_t$, as illustrated in Fig. 2. To prevent potential bias in decision-making that can arise from directly inputting raw state values into the network, we apply *min-max* normalization to scale the state vector. This ensures that all state values are proportionally represented, improving both the fairness and accuracy of the agent's decisions.

▷ **Action Space.** The action space $\mathcal{A}$ is defined as the predicted cwnd for the next state s_{t+1} . For each time step t , the AERO policy π^* maps the $\vec{s}_t$ to a continuous action space $\mathcal{A}$ and selects $a_t \in \mathcal{A}$.

▷ **Output.** The actor outputs a scalar α used as a multiplier for the next congestion window (cwnd), with policy $\pi^* : s_t \rightarrow a_t^*$; the critic outputs the value $V^\pi(s_t)$. The next window is $\text{cwnd}_{t+1} = A^\alpha \times \text{cwnd}_t$, where A is the base multiplier and α the power factor. We chose this multiplicative parametrization over alternative formulations for three reasons. First, compared to directly predicting an absolute cwnd value, the multiplicative form $A^\alpha \times \text{cwnd}_t$ preserves continuity with the current congestion state, preventing abrupt window jumps that could destabilize the connection. Second, the base-power structure A^α with $\alpha \in [-1, 1]$ naturally encodes both increase ($\alpha > 0$) and decrease ($\alpha < 0$) within a single continuous action, simplifying the policy output to a single scalar while still covering a wide dynamic range. Third, unlike additive formulations (e.g., $\text{cwnd} \pm \delta$), the multiplicative form scales proportionally with the current cwnd, ensuring that adjustments are appropriately sized regardless of whether the connection operates at low or high throughput regimes. For pacing, AERO follows the standard QUIC pacing mechanism [26], where the pacing rate is computed as cwnd/SRTT . Since AERO only modifies the cwnd update logic, the underlying pacing behavior remains consistent with the host QUIC implementation. A is a learned parameter of AERO, bounded in $[1.2, 3]$. Algorithm 1 details how AERO updates the reward r and, consequently, A ; we report the average A across all training trajectories. The update rules in Algorithm 1 are designed to ensure stable policy learning and to prevent oscillatory congestion window dynamics. The use of two deviation regimes—a *small deviation* region ($\Delta \text{cwnd} < \phi$) and a *large deviation* region ($\Delta \text{cwnd} \geq \phi$)—is motivated by standard proportional control principles, where the correction magnitude increases with the deviation from the target operating point [4]. This avoids either overreacting to small deviations or underreacting to large ones. Accordingly, the ρ and 2ρ increments provide a coarse but bounded proportional step size, which prevents divergence while enabling responsive adaptation. Moreover, policy gradient methods such as PPO and A2C are known to be highly sensitive [14] to reward scale [19] and variance [45, 50]. Large or unbounded reward fluctuations can destabilize training by inducing excessively large gradient updates [3]. By constraining updates to ρ -scaled adjustments, Algorithm 1 maintains a well-conditioned reward range and prevents abrupt parameter shifts. Finally, adjusting the base factor A incrementally ensures smooth multiplicative cwnd scaling. This is consistent with classical congestion control design, where gradual increase and decisive decrease prevent window

oscillation and ensure stable convergence toward the available capacity [25, 27]. Thus, although simple, the $\rho/2\rho$ structure is not ad-hoc: it enforces (i) bounded gradient magnitudes, (ii) deviation-proportional corrections, and (iii) stable cwnd evolution.

▷ **NN Architecture.** The AERO NN architecture (Fig. 2) consists of two networks: the actor and the critic. Each network uses three fully connected (FC) layers, with the actor network being augmented by two batch normalization layers for heightened training stability. Each FC layer contains 256 neurons and utilizes Leaky Rectified Linear Unit (Leaky ReLU()) activation function. The output of the last FC layer in the actor network uses $\tanh$ activation function, providing an output $\in [-1, 1]$. In the critic network $\vec{s}_t$ and a_t are concatenated in the first FC layer for each time step t . The output is then derived from the last FC layer.

▷ **Reward Function.** At each time step t , the agent takes an action a_t and receives a corresponding reward r_t . This reward evaluates the quality of the action a_t taken in the current state s_t , guiding the agent toward learning the optimal policy π^* . For AERO, we design a simple yet effective reward function that linearly combines four metrics: throughput (th_t), delay (d_t), packet loss ratio (l_t), and jitter (j_t),

$$r_t = r_t + th_t - w_1 \times d_t - w_2 \times l_t - w_3 \times j_t, \quad (2)$$

Initially, r_t is modified according to Algorithm 1, and the final reward is computed based on the current state s_t . To achieve the best trade-off among the four metrics, we used a small subset (10%) of the training dataset for tuning. Our results indicate the following weighting scheme: $w_3 = 2$ in all scenarios; when the proportion of video packets is highest, $w_1 = 4$, $w_2 = 5$; when audio packets dominate, $w_2 = 4$; and when probing packets dominate, $w_2 = 2$ and $w_1 = 3$. This weighting reflects the fact that packet loss impacts video traffic more significantly than probing packets. In the online setting, we determine the dominant packet type using only the segment size information available in the manifest file (MPD). The MPD specifies the minimum and maximum segment sizes for video representations, which provides a natural range for identifying video segments. Segments that fall near this range (e.g., close to the maximum video representation size) are treated as video packets. Audio segments are typically an order of magnitude smaller and appear consistently in the kilobyte range. Probing or control-related transfers are even smaller and appear as very short exchanges. Thus, by comparing the requested chunk size at time t to the minimum and maximum segment sizes indicated in the MPD, we can reliably infer whether the ongoing traffic is video, audio, or probing, without requiring packet-level labeling or protocol modifications. To ensure that AERO is not overly dependent on this adaptive weighting, we also evaluate a version of the model that uses a single fixed set of weights (i.e., without adjusting based on segment type). As shown in Appendix D, this fixed-weight version remains stable and learns a functional congestion control policy, but achieves lower QoE and less efficient throughput–delay trade-offs compared to the adaptive scheme. We emphasize that AERO’s core architecture operates purely at the transport layer, relying only on four application-agnostic signals: throughput, delay, packet loss, and jitter. The adaptive weighting scheme described above is an optional enhancement that leverages segment size information available in the HAS manifest file (MPD) to improve QoE for video streaming workloads. When this information is unavailable, such as in non-streaming applications, AERO defaults to a fixed set of weights. As shown in Appendix D, this fixed-weight version remains stable and learns a functional congestion control policy, confirming that the adaptive weighting is not a prerequisite for AERO’s operation. The underlying DRL architecture and cwnd update mechanism generalize to any QUIC-based application, as the transport-layer signals on which AERO’s policy is trained are observed regardless of the application workload.

▷ **Policy Gradient and Training Algorithm.** The primary goal of AERO is to refine policy strategies by increasing the probabilities of high-reward actions from compiled trajectories while reducing the influence of sub-optimal actions. At each time step t , the agent maps the state vector $\vec{s}_t$ to the optimal multiplicative factor α_t to compute the next congestion window for state s_{t+1} using the policy π :

$$\pi_\theta^*(\alpha_t, a_t) \rightarrow [-1, 1] \text{ at state } s_t,$$

Algorithm 1 AERO reward and cwnd base factor update.

```

1: procedure UPDATE( $\hat{cwnd}_t, E[cwnd_t], cwnd_{max}, r_t, \phi, \rho, A$ )
   //  $\hat{cwnd}_t$ : predicted cwnd for  $s_t$  from  $s_{t-1}$ ;  $E[cwnd_t]$ : expected cwnd for  $s_t$ 
   //  $cwnd_{max}$ : max allowed cwnd;  $r_t$ : reward at step  $t$ ;  $\phi$ :  $\Delta cwnd$  threshold
   //  $\rho$ : gain/penalty factor ( $= 0.1$ );  $A$ : base factor ( $cwnd_{t+1} = A^\alpha \times cwnd_t$ )
2:  $r_t \leftarrow 0$ ; estimate  $E[cwnd_t]$  from bottleneck capacity;  $\Delta cwnd \leftarrow |\hat{cwnd}_t - E[cwnd_t]|$ 
3: if  $\hat{cwnd}_t > cwnd_{max}$  then  $r_t \leftarrow r_t - 2\rho$ ,  $A \leftarrow A - \rho$  ▷ Exceeds max
4: end if
5: if  $\hat{cwnd}_t = E[cwnd_t]$  then  $r_t \leftarrow r_t + 1.5\rho$  ▷ Exact match
6: else if  $\hat{cwnd}_t > E[cwnd_t]$  then ▷ Over-estimated
7:   if  $\Delta cwnd < \phi$  then  $r_t \leftarrow r_t - \rho$ ,  $A \leftarrow A - \rho/2$  ▷ Small deviation
8:   else  $r_t \leftarrow r_t - 2\rho$ ,  $A \leftarrow A - \rho$  ▷ Large deviation
9:   end if
10: else ▷ Under-estimated
11:   if  $\Delta cwnd < \phi$  then  $r_t \leftarrow r_t + \rho$ ,  $A \leftarrow A + \rho/2$  ▷ Small deviation
12:   else  $r_t \leftarrow r_t + 2\rho$ ,  $A \leftarrow A + \rho$  ▷ Large deviation
13:   end if
14: end if
15: end procedure

```

which maximizes the advantage function:

$$\Lambda_t = \sum_{i=t}^{T_{\pi_\theta}} r_i + \gamma \delta_t + \gamma \lambda \Lambda_{t+1}, \quad (3)$$

$$a_t = \arg \max_a E[\Lambda_t(\alpha_t, a)],$$

where Λ_t is computed for each time step t until the end of training, T_{π_θ} denotes the batch size for policy gradient updates, $\gamma \in [0, 1]$ is the discount factor, and $\delta_t = V(s_{t+1}) - V(s_t)$ is the temporal difference (TD) error. Here, $V(s_t)$ is the value function predicted by the critic network, λ is the GAE parameter, θ are the policy parameters, and $\pi_\theta(\alpha_t, a_t)$ is the probability of taking action a_t in state s_t . The TD error provides immediate feedback on deviations between predicted and observed values, while GAE improves advantage estimation by incorporating both short-term and long-term effects of actions, enabling more informed and stable policy updates.

After collecting an experience trajectory, PPO computes mini-batch updates using a clipped surrogate objective function to ensure stable and efficient policy learning:

$$L^{clip}(\theta) = -E_t \left[\min \left(\mathfrak{R}_t(\theta) \cdot \hat{\Lambda}_t, \text{clip}(\mathfrak{R}_t(\theta), 1 - \epsilon, 1 + \epsilon) \cdot \hat{\Lambda}_t \right) \right], \quad (4)$$

where $L^{clip}(\theta)$ is the clipped surrogate objective (policy loss), $\mathfrak{R}_t(\theta)$ is the probability ratio of the new to old policy, $\hat{\Lambda}_t$ is the estimated advantage at time t , and ϵ controls the size of the policy update. The negative sign indicates minimization to achieve stable, incremental improvements.

The value function loss is computed as:

$$L^{value}(\theta) = \frac{1}{2} E_t \left[(V(s_t; \theta) - G_t)^2 \right], \quad (5)$$

where $V(s_t; \theta)$ is the critic's predicted value for s_t , and G_t is the observed return.

We also compute an entropy loss to encourage exploration:

$$L^{ent}(\theta) = E_t \left[\sum_a \pi(a|s_t; \theta) \log \pi(a|s_t; \theta) \right]. \quad (6)$$

The total gradient is then computed as:

$$\nabla_\theta L^{total}(\theta) = \nabla_\theta L^{clip}(\theta) + \nabla_\theta L^{value}(\theta) + \nabla_\theta L^{ent}(\theta), \quad (7)$$

and parameters are updated using gradient descent:

$$\theta \leftarrow \theta - \omega \nabla_{\theta} L^{total}(\theta), \quad (8)$$

where ω is the learning rate for the actor/critic networks. This procedure is repeated for each epoch, ultimately yielding the optimal policy π^* .

All symbols used in AERO DRL training are listed in Appendix B. Additionally, we conducted extensive evaluation studies to refine the DRL design of AERO. For details on hyperparameter tuning and DRL ablation studies, please refer to Appendix D. We also measured the computational usage of AERO; further details are provided in Appendix E.

3.3 Pluggable Design

A key design goal of AERO is seamless integration into any QUIC implementation without extensive engineering effort. Three architectural properties enable this pluggability. First, AERO operates entirely in user space, requiring no kernel-level modifications. This aligns with QUIC’s fundamental design principle of implementing congestion control outside the kernel [26], and ensures that AERO can be deployed alongside existing QUIC stacks without altering the host operating system or requiring elevated privileges. Second, AERO’s input interface relies exclusively on four transport-layer signals—throughput, delay, packet loss ratio, and jitter—that are universally exposed by QUIC implementations through their congestion controller class or equivalent module. No additional instrumentation, custom hooks, or protocol extensions are required. For instance, Quiche exposes these metrics via its `CongestionControl` trait in Rust, Quinn through its `CongestionController` trait, Aioquic via its `QuicCongestionControl` class in Python, and Tquic through its `CongestionController` trait. In each case, the required signals are already computed and available within the existing CC interface. Third, AERO’s output is a single scalar cwnd update of the form $cwnd_{t+1} = A^{\alpha} \times cwnd_t$, which replaces only the cwnd adjustment logic within the congestion controller. This multiplicative update is a universal primitive: every QUIC CC module maintains and modifies a cwnd variable, making AERO a drop-in replacement for the update rule without restructuring the surrounding loss detection, recovery, or pacing logic.

From an engineering perspective, integrating AERO into a new QUIC stack involves three steps. First, the four input signals are read from the congestion controller’s state variables, which are maintained on a per-connection basis in all QUIC implementations (e.g., `bytes_in_flight`, `latest_rtt`, `lost_packets`, `rtt_variance`). Second, a lightweight inference call is made to the trained AERO model, which can be loaded via a shared library or an inter-process socket depending on the implementation language. Third, the resulting cwnd value is written back to the connection’s congestion state. No changes to the QUIC handshake, packet parsing, loss detection, or stream management logic are required. We validate this in Section 4.3.3 by integrating AERO into four QUIC implementations written in three different programming languages (Rust, Go, and Python).

4 Evaluation and Results

4.1 AERO Implementation

4.1.1 AERO Parameter Selection. AERO was trained on a dataset [39] comprising 9,405 audio/video calls, covering a wide range of network conditions as described in Section 4.2.2. Since the training and testing platforms were distinct, all available traces were employed exclusively for training. Hyperparameter tuning plays a critical role in optimizing any neural network architecture; therefore, we used Bayesian optimization techniques to identify the optimal parameters for the AERO model. The resulting optimal hyperparameter set is summarized in Table 1.

4.1.2 Offline Training. AERO’s NN architecture was implemented in Pytorch 2.1.1+cu121 with Python 3.10.12. The training was done in a continuous trace driven fashion as highlighted in Section 3.2.

4.1.3 Online Testing. For cwnd updates during AERO testing, we used the average value of the base multiplier A derived from training-phase experience. The average A was 1.87. During testing, the min-max bounds for cwnd were set according to the lowest and highest bandwidth specified in the respective manifest file (HAS MPD). To evaluate AERO, we employed the *Vegvisir* framework [20] for QUIC trace-driven simulations. Within *Vegvisir*, we implemented a Quiche-based QUIC server and a Quic-go-based QUIC client, as shown in Fig. 3. We simulated typical HAS video streaming sessions using multiple video contents under diverse network conditions. The Quic-go client leveraged a goDASH-based player [43] in headless mode, allowing straightforward extraction of metrics while avoiding the complexities of web-based dash.js players. For network emulation, we utilized tc-Netem [18] with multiple traces, as illustrated in Fig. 3.

4.2 Methodology

4.2.1 Video Sample and Parameters. *3DMark Night Raid* (NR) [52] and *Seconds That Count* (STC) [52] video contents were used during AERO testing, featuring different segment durations and maximum buffer capacities. Notably, all videos support up to 4K resolution. All videos are encoded in AVC format with 10 bitrate levels ranging between 288p@0.5Mbps–2160p@40Mbps (4K). We evaluated AERO’s performance across two streaming modes: video-on-demand (VoD) and low-latency live (LLL), with multiple scenarios defined for each mode. Details on the video contents and streaming scenarios are provided in Table 2. To ensure robustness, all trace-driven simulations were repeated three times, with each run lasting approximately 120 seconds, and the results reported represent the average across these runs. All streaming sessions use a single persistent QUIC connection per client, with cwnd state preserved across segment downloads, consistent with standard HAS behavior over QUIC.

4.2.2 Network Traces. We used three network traces: 5G Amazon [42], 5G Netflix [42], and LTE Belgium [54], with average bandwidths (and sampling intervals) of 24.9Mbps (1s), 33.3Mbps (1s), and 20Mbps (1s), respectively. Note that the 5G traces correspond to sub-6 GHz deployments, not mmWave.

4.2.3 CC Algorithms and Metrics. We used several heuristic-based CC implementations available in Quiche for comparison. For loss-based algorithms, Cubic and Reno served as baselines, while BBR2 was used for hybrid CC. Although Quiche also provides BBR, we did not include it in the comparison since BBR2 consistently outperforms it. Most existing learning-based CC algorithms are implemented for TCP and require kernel patches, which contradicts the goal of having a user-mode CC for QUIC; therefore, they were not considered as baselines. To evaluate AERO against these CC algorithms, we employed two key metrics: mean normalized delivery rate and delay. The values for delivery rate and delay were extracted directly from their respective qlog [38] files and linearly interpolated over time. Formally, let $DR_i(t)$ and $d_i(t)$ denote the delivery rate and delay of CC algorithm i at time t , extracted from the respective qlog files and linearly interpolated over time. The mean delivery rate and mean delay for algorithm i are computed as:

$$\overline{DR}_i = \frac{1}{T} \sum_{t=1}^T DR_i(t), \quad \bar{d}_i = \frac{1}{T} \sum_{t=1}^T d_i(t), \quad (9)$$

where T is the total number of time samples. To enable comparison across traces with different absolute scales, we apply min-max normalization across all CC algorithms for a given trace:

$$\hat{DR}_i = \frac{\overline{DR}_i - \min_j(\overline{DR}_j)}{\max_j(\overline{DR}_j) - \min_j(\overline{DR}_j)}, \quad \hat{d}_i = \frac{\bar{d}_i - \min_j(\bar{d}_j)}{\max_j(\bar{d}_j) - \min_j(\bar{d}_j)}, \quad (10)$$

where j ranges over all CC algorithms evaluated for that trace. The normalized values $\hat{DR}_i, \hat{d}_i \in [0, 1]$ are then scaled to $[0, 2]$ for visualization purposes. In Fig. 4, each marker represents the mean across three experimental runs, and the error bars denote ± 1 standard deviation, capturing the variability across repetitions. To provide

Table 1. AERO training/testing parameters.

	Parameter	Symbol	Value
A2C	Discount Factor	γ	0.95
	Advantage estimator	λ	0.9
	Actor & Critic learning rates	$\omega, \bar{\omega}$	$3 \times 10^{-4}, 3 \times 10^{-3}$
PPO	Clip parameter	ϵ	0.2
	Entropy Coefficient	δ	2.6
	PPO policy update interval, steps	T_θ, n_s	725, 6
NN	Batch size	B_s	128
	Exploration size, number of episodes	θ, n_e	29, 25
	Number of epochs	itr	38000

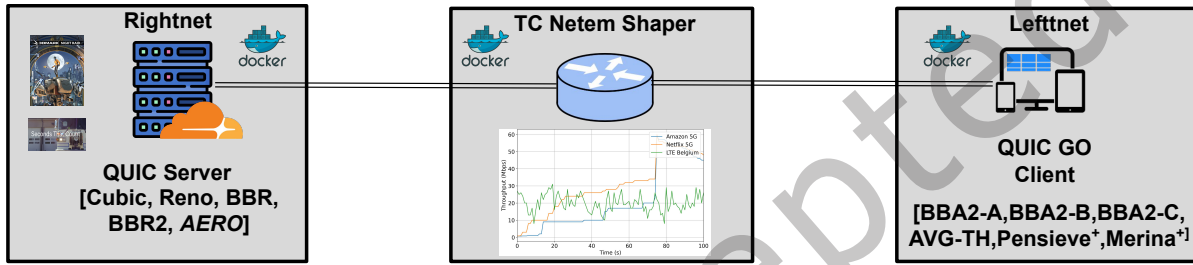


Fig. 3. Vegvisir framework.

Table 2. Details of experimental scenarios.

Mode	Scen.	Video	FPS	Bitrate (Mbps)	Seg. Dur. (s)	Max Buff. (s)
VoD	A	NR	60	0.5 - 40	4	60
	B	STC	24			
LLL	C	NR	60	0.5 - 40	2	12
	D	STC	24			

a more detailed distributional view, violin plots of the normalized delivery rate and delay are presented in Appendix F.

4.2.4 ABR Algorithms and Metrics. We employed two heuristic-based and two learning-based ABR schemes. Among the heuristic-based schemes, we used one buffer-based, BBA2-C, and one throughput-based, AVG-TH, both from [21]. BBA2-C incorporates an advanced stall-prevention mechanism to maintain seamless streaming. It anticipates potential stalls by evaluating the download of both the current segment and the subsequent one at a bitrate lower than the current selection. If a stall is predicted, BBA2-C cancels the current request and initiates a new request for the segment at the lowest bitrate. This proactive approach mitigates buffer depletion before stalls occur, enhancing overall streaming performance. AVG-TH, on the other hand, computes the average of the most recent 20 throughput estimates to make ABR decisions. For the learning-based ABR schemes, we used Pensieve [37] and Merina [29]. Originally, both Pensieve and Merina were trained to select among six bitrate representations. Since our video samples have 10 bitrate levels, we re-trained the models for 10 bitrates over more epochs. Additionally, we implemented the stall-prevention logic from BBA2-C into Pensieve (denoted as Pensieve⁺) and Merina (denoted as Merina⁺).

To enable a comprehensive comparison across all combinations of CC and ABR algorithms, we employed four well-known QoE models: QoE^{itu} (0.46) [24], Duanmu [13], Yin [59], and Yu [60]. These QoE models incorporate two or more metrics as input, such as the selected bitrate, number of rebuffering events (nReb), rebuffering ratio

(RD), number of segments downloaded in ultra-HD quality (nSeg4K), startup delay, and bitrate switches, each according to the model's objective. The output of each model is a QoE value ranging between 1 (bad) and 5 (good) for QoE^{itu} , and normalized between 0 (bad) and 1 (good) for the remaining models. Additionally, we evaluated perceptual quality using VMAF [32] and PSNR [51].

4.2.5 Experimental Setup. Our setup comprised a physical machine running Ubuntu 22.04.4 LTS, equipped with an NVIDIA GeForce RTX 3060 GPU, a 24-core CPU, and 64 GB of memory. During the testing phase, the client, shaper, and server were each encapsulated within Docker containers, forming a cohesive streaming system. These containers were interconnected via two Docker-compose networks, configured to route traffic through the shaper, thereby effectively managing and regulating the flow of traffic between them, as illustrated in Fig. 3.

4.3 Results and Analysis

4.3.1 AERO CC for QUIC Results. In this section, we compare the AERO CC efficiency against heuristic-based CC algorithms. Fig.4 shows the normalized mean delivery rate versus normalized mean delay for AERO, BBR2, Cubic, and Reno, in different network traces, and for LLL (top three Figures) and VoD (bottom three Figures) streaming modes.

▷ *LLL Mode.* In the 5G Amazon trace (top left), AERO, BBR2, and Reno exhibit comparable delay, whereas Cubic experiences the highest delay. However, in terms of delivery rate, AERO significantly outperforms all other CC algorithms. Similar trends are observed in the 5G Netflix trace (top center) and the LTE Belgium trace (top right), with Reno showing the highest delay instead of Cubic for LTE Belgium. The limited buffer capacity in LLL mode amplifies the impact of latency variations on BBR2's CC decisions, potentially leading to sub-optimal delivery rates. Cubic and Reno, being purely loss-based, aggressively increase their delivery rate until a packet loss occurs, causing frequent oscillations between high and low delivery rates and resulting in inconsistent performance across different network conditions. In contrast, AERO demonstrates consistently superior performance across all traces. Overall, AERO achieves approximately 161% higher average delivery rate compared to BBR2, while only incurring a slightly higher delay (around 18%). As further validated in Section 4.3.2, both delay and delivery rate are critical for achieving better QoE in LLL mode.

▷ *VoD Mode.* AERO consistently achieves the highest delivery rate across all traces while closely mirroring BBR2 and Reno in terms of delay. While BBR2 manages to provide the least delay across all traces, it significantly lags behind in terms of delivery rate. BBR2's heightened sensitivity to latency remains the primary factor behind its consistently lowest delivery rate across all traces. Reno indicates a better performance than BBR2 in terms of delivery rate but lags behind in delay due to its aggressive and conservative nature. Cubic, being even more aggressive than Reno, is the worst performing CC both in terms of delivery rate and delay. Overall, AERO is able to attain a ~245% higher delivery rate in comparison to BBR2 while only having a slightly higher (~12%) delay. As further substantiated by our findings in Section 4.3.2, delivery rate is the most critical factor for VoD streaming, a criterion that AERO successfully meets.

These results indicate that the existing heuristic-based CC algorithms lack adaptability to varying network conditions and have different behavior for varying video streaming modes, which impacts the QoE negatively as shown in Table 3 to Table 6. In contrast, AERO performs consistently across all, highlighting its generalizability and good performance.

4.3.2 Impact of AERO on ABR over QUIC Results. In this section, we evaluate the impact of AERO CC on both heuristic-based and learning-based ABR schemes. Tables 3 and 4 present the comparison of various QoE metrics across different network traces in LLL mode, while Tables 5 and 6 summarize the results for VoD mode.

▷ *LLL Mode.*

1) Scenario C. The results for Scenario C (refer Table 2) are summarized in Table 3. Across all traces, AERO+Pensieve⁺ achieves the highest average bitrate (col. 3) while maintaining minimal rebuffering (col. 4). In the 5G Amazon

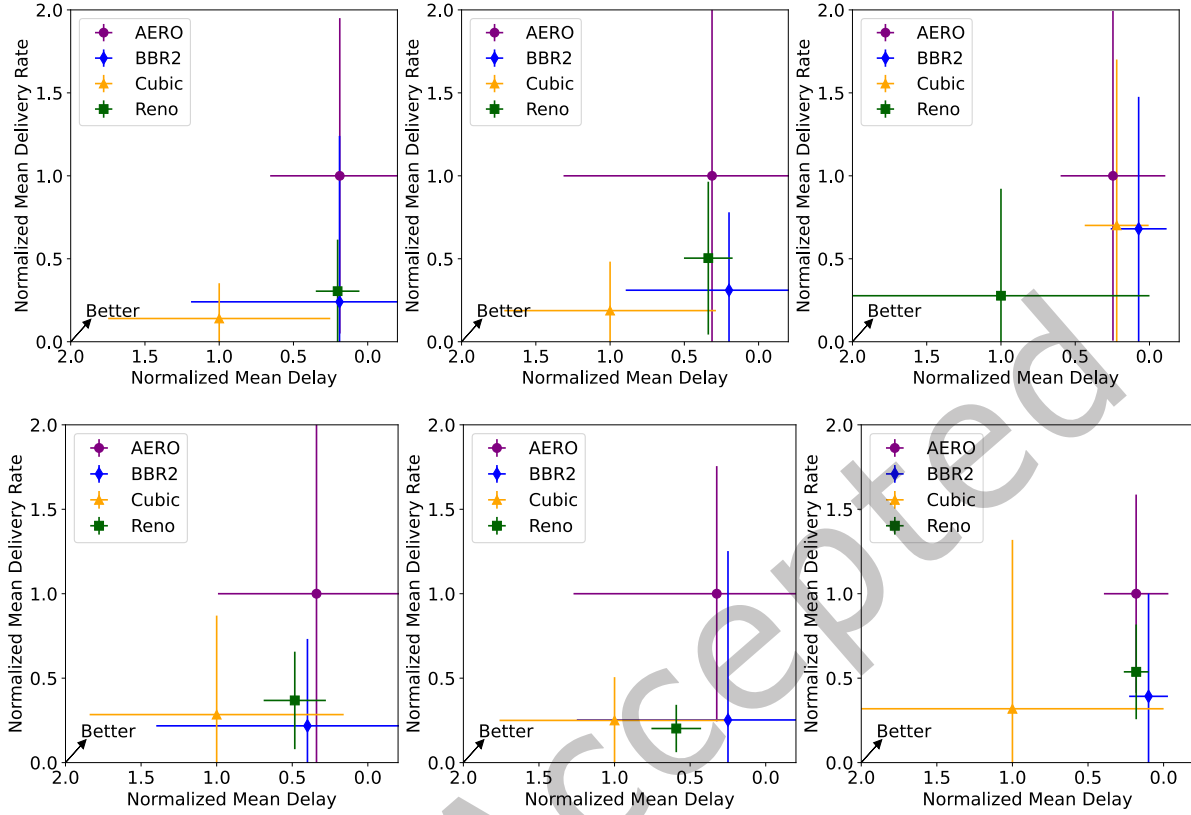


Fig. 4. Normalized delivery rate vs delay in LLL (top three) and VoD (bottom three) for network traces 5G Amazon (left), 5G Netflix (center) and LTE Belgium (right), respectively.

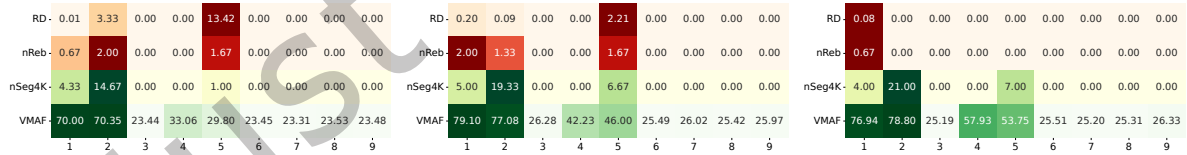


Fig. 5. VMAF vs nReb, nSeg4K, RD analysis in LLL for network traces 5G Amazon (left), 5G Netflix (center), and LTE Belgium (right). 1: AERO+Pensieve⁺, 2: AERO+Merina⁺, 3: AERO+BBA2-C, 4: BBR2+Pensieve⁺, 5: BBR2+Merina⁺, 6: BBR2+BBA2-C, 7: Cubic+BBA2-C, 8: Reno+BBA2-C, 9: BBR2+AVG-TH.

trace, AERO+Merina⁺ attains the highest VMAF (col. 1), approximately 135% higher than its counterpart, well above the Just Noticeable Difference (JND) [11]¹—a substantial improvement for streaming 4K content. Although BBR2+Merina⁺ experiences fewer rebuffering events compared to its counterpart, the duration of these stalls is notably higher, as shown in Fig. 5.

¹A difference of 6 VMAF units is widely recognized as the JND threshold, beyond which a typical viewer can perceive a noticeable quality change.

Table 3. Average results of the QoE and its metrics for different network traces for scenario C. ↑: higher is better (green), ↓: lower is better (green), lowest performance (red), and mode: LLL.

		Avg. Perceptual Quality (VMAF)	Avg. Perceptual Quality (PSNR)	Avg. Selected Bitrate (Mbps)	Average RD (%)	Average Duanmu	Average Yu	Average Yin	Avg. QoE ^{itu} (0.46)
5G Amazon	AERO+Pensieve ⁺	70.00	38.91	3.94 ↑	0.01	1.00 ↑	1.00 ↑	1.00 ↑	2.72 ↑
	AERO+Merina ⁺	70.35 ↑	39.94 ↑	2.90	3.33	0.24	0.61	0.16	2.00
	AERO+BBA2-C	23.44	32.00	0.54	0.00 ↓	0.59	0.16	0.27	1.87
	BBR2+Pensieve ⁺	33.06	33.06	0.74	0.00 ↓	0.62	0.18	0.12	1.90
	BBR2+Merina ⁺	29.80	32.77	0.55	13.42	0.19	0.17	0.14	1.77
	BBR2+BBA2-C	23.45	31.79	0.52	0.00 ↓	0.59	0.16	0.26	1.87
	Cubic+BBA2-C	23.31	31.93	0.52	0.00 ↓	0.59	0.16	0.28	1.87
	Reno+BBA2-C	23.53	31.92	0.50	0.00 ↓	0.58	0.15	0.28	1.87
	BBR2+AVG-TH	23.48	31.91	0.63	0.00 ↓	0.61	0.18	0.27	1.87
5G Netflix	AERO+Pensieve ⁺	79.10 ↑	40.64	6.65 ↑	0.20	1.00 ↑	1.00 ↑	0.32	1.55
	AERO+Merina ⁺	77.08	41.43 ↑	4.40	0.09	0.89	0.58	0.00	1.88
	AERO+BBA2-C	26.28	32.37	1.62	0.00 ↓	0.61	0.30	0.78	1.89
	BBR2+Pensieve ⁺	42.23	34.21	1.24	0.00 ↓	0.59	0.20	0.21	1.99 ↑
	BBR2+Merina ⁺	46.00	35.63	1.10	2.21	0.26	0.17	0.06	1.85
	BBR2+BBA2-C	25.49	32.09	1.09	0.00 ↓	0.56	0.20	0.55	1.88
	Cubic+BBA2-C	26.02	32.27	1.09	0.00 ↓	0.57	0.19	0.44	1.88
	Reno+BBA2-C	25.42	32.13	1.02	0.00 ↓	0.56	0.17	0.31	1.88
	BBR2+AVG-TH	25.97	32.43	1.23	0.00 ↓	0.57	0.26	1.00 ↑	1.88
LTE Belgium	AERO+Pensieve ⁺	76.94	39.89	5.68 ↑	0.08	1.00 ↑	1.00 ↑	0.00	1.33
	AERO+Merina ⁺	78.80 ↑	41.40 ↑	3.24	0.00 ↓	0.82	0.54	0.00	1.69
	AERO+BBA2-C	25.19	32.28	1.38	0.00 ↓	0.59	0.39	1.00 ↑	1.89
	BBR2+Pensieve ⁺	57.93	36.47	2.76	0.00 ↓	0.77	0.58	0.00	1.81
	BBR2+Merina ⁺	53.75	36.56	1.00	0.00 ↓	0.56	0.24	0.29	1.92 ↑
	BBR2+BBA2-C	25.51	32.13	1.43	0.00 ↓	0.60	0.40	0.92	1.89
	Cubic+BBA2-C	25.20	32.27	1.39	0.00 ↓	0.59	0.39	0.99	1.89
	Reno+BBA2-C	25.31	32.20	1.41	0.00 ↓	0.59	0.39	0.96	1.89
	BBR2+AVG-TH	26.33	32.17	2.56	0.00 ↓	0.77	0.57	0.46	1.90

AERO+Pensieve⁺ (runner-up) also exhibits significant VMAF improvement *w.r.t.* its counterpart and achieves the highest average QoE^{itu} (0.46, col. 8). Notably, QoE^{itu} (0.46) P.1203 is limited to resolutions up to 1080p while the extended ITU-T P.1204 implementation for 4K content remains in its early development stage. It lacks frame-by-frame parsing capability and is limited to short video sequences of up to 10 seconds [9, 48]. This makes QoE^{itu} (0.46) an unreliable metric for 4K videos. AERO+BBA2-C performs comparably to BBR2+BBA2-C on all metrics. Cubic, Reno, and BBR2, when combined with BBA2-C, as well as BBR2+AVG-TH, show comparable performance but fall significantly short in VMAF. Similar trends are observed for the 5G Netflix trace, where AERO+Pensieve⁺ achieves the highest VMAF. Although BBR2+Pensieve⁺ achieves the highest average QoE^{itu} (0.46), it lags significantly in VMAF compared to its counterpart. The results for the LTE Belgium trace are consistent with the 5G traces.

Fig. 5 additionally highlights the number of segments downloaded in 4K resolution (nSeg4K) across all network traces for the given combinations. AERO+Merina⁺ and AERO+Pensieve⁺ consistently download more high-quality segments over the same duration, reflecting their substantially higher average bitrates. In contrast, BBR2 combined with the same ABRs downloads fewer segments due to its lower average bitrate.

2) Scenario D. The results for this mode in Scenario D (see Table 2) are summarized in Table 4. Here, clients stream a separate 4K video (STC) at 24fps, in contrast to the 60fps video (NR) used in Scenario C. This difference in video content and frame rate provides complementary conditions for evaluating CC algorithms: while Scenario C stresses the system with high-frame-rate video, Scenario D tests its adaptability to heterogeneous video characteristics and typical streaming workloads. Together, these scenarios offer a more comprehensive assessment of the CC's performance across diverse streaming conditions.

Table 4. Average results of the QoE and its metrics for different network traces for scenario D. ↑: higher is better (green), ↓: lower is better (green), lowest performance (red), and mode: LLL.

		Avg. Perceptual Quality (VMAF)	Avg. Perceptual Quality (PSNR)	Avg. Selected Bitrate (Mbps)	Average RD (%)	Average Duanmu	Average Yu	Average Yin	Avg. QoE ^{itu} (0.46)
5G Amazon	AERO+Pensieve ⁺	83.23	48.69	2.53	0.08	0.91	0.91	1.00 ↑	2.29 ↑
	AERO+Merina ⁺	87.08 ↑	50.30 ↑	2.88 ↑	0.16	1.00 ↑	0.99 ↑	0.12	2.12
	AERO+BBA2-C	42.66	40.11	0.87	0.00 ↓	0.95	0.81	0.98	1.87
	BBR2+Pensieve ⁺	50.74	41.58	1.33	0.00 ↓	0.84	0.48	0.51	1.90
	BBR2+Merina ⁺	54.18	42.41	1.29	1.00	0.67	0.51	1.00 ↑	1.82
	BBR2+BBA2-C	43.80	40.33	1.10	0.00 ↓	1.00 ↑	1.00 ↑	0.87	1.89
	Cubic+BBA2-C	43.72	40.39	1.06	0.00 ↓	0.99	0.97	1.00 ↑	1.88
	Reno+BBA2-C	43.66	40.38	1.07	0.00 ↓	0.99	0.97	0.98	1.88
	BBR2+AVG-TH	41.86	39.96	0.74	0.00 ↓	0.66	0.63	1.00 ↑	1.87
5G Netflix	AERO+Pensieve ⁺	88.43	50.72	6.72 ↑	0.03	1.00 ↑	0.99	0.69	1.58
	AERO+Merina ⁺	89.92 ↑	51.71 ↑	5.36	0.41	0.95	0.96	0.11	1.55
	AERO+BBA2-C	46.77	40.71	1.88	0.00 ↓	1.00 ↑	1.00 ↑	1.00 ↑	1.89
	BBR2+Pensieve ⁺	66.43	44.18	3.48	0.01	0.76	0.51	0.68	2.00 ↑
	BBR2+Merina ⁺	71.29	47.07	3.59	2.09	0.75	0.63	0.10	1.91
	BBR2+BBA2-C	45.43	40.58	1.71	0.00 ↓	0.99	0.93	0.96	1.90
	Cubic+BBA2-C	44.65	40.46	1.28	0.00 ↓	0.91	0.70	0.84	1.88
	Reno+BBA2-C	43.94	40.45	1.21	0.00 ↓	0.90	0.66	0.81	1.88
	BBR2+AVG-TH	43.50	40.70	1.26	0.00 ↓	0.81	0.83	1.00 ↑	1.88
LTE Belgium	AERO+Pensieve ⁺	88.01 ↑	49.77	10.40 ↑	0.00 ↓	1.00 ↑	1.00 ↑	0.63	1.36
	AERO+Merina ⁺	86.74	50.70 ↑	9.79	0.00 ↓	1.00 ↑	1.00 ↑	1.00 ↑	2.12 ↑
	AERO+BBA2-C	44.08	40.13	1.53	0.00 ↓	0.98	0.96	0.88	1.89
	BBR2+Pensieve ⁺	66.36	44.87	6.52	5.48	0.34	0.57	0.05	1.91
	BBR2+Merina ⁺	75.82	46.81	6.30	0.49	0.56	0.52	0.16	1.88
	BBR2+BBA2-C	44.74	40.35	1.64	0.00 ↓	1.00 ↑	1.00 ↑	0.78	1.89
	Cubic+BBA2-C	44.22	40.25	1.55	0.00 ↓	0.98	0.97	0.86	1.89
	Reno+BBA2-C	43.31	40.16	1.39	0.00 ↓	0.94	0.92	1.00 ↑	1.88
	BBR2+AVG-TH	44.50	40.94	2.69	0.02	0.94	0.62	0.12	1.88

The results in Scenario D closely mirror those observed in Scenario C. Across all traces, AERO combined with Pensieve⁺ or Merina⁺ achieves the highest VMAF (col.1), approximately 31% higher than competing approaches, along with the highest PSNR (col.2) and average bitrate (col.3), while maintaining minimal rebuffering (col.4). This combination ensures high QoE and a seamless playback experience—a performance that other CC algorithms (BBR2, Cubic, and Reno) fail to match, falling significantly behind in both perceptual quality and bitrate efficiency.

Overall, AERO+Pensieve⁺ and AERO+Merina⁺ achieve the highest VMAF scores while maintaining minimal rebuffering. Their counterparts lag significantly in VMAF by ~70%. Additionally, these combinations select the highest average bitrate along with the highest PSNR (col. 2), providing a consistently smooth streaming experience. The QoE^{itu} (0.46) proves unreliable for these scenarios as they are streaming 4K content. Across all metrics, AERO+BBA2-C performs comparably *w.r.t.* BBR2, Cubic, and Reno when combined with the same ABR baseline. In contrast, BBR2+Merina⁺ experiences the longest rebuffering durations, negatively impacting QoE. These observations align with our findings that AERO maintains a substantially higher delivery rate with minimal delay, thereby enhancing ABR decision-making and highlighting AERO's adaptability across all network traces. Importantly, the consistent performance across both high-frame-rate (60 fps) and standard-frame-rate (24 fps) videos underscores AERO's robustness to diverse video content and heterogeneous streaming workloads.

▷ VoD Mode.

1) Scenario A. As shown in Table 5, across all traces, AERO+BBA2-C provides a rebuffering free playback with comparable average bitrate (col.3) and it trails slightly behind AERO+Pensieve⁺ and AERO+Merina⁺ in terms of VMAF. In contrast, BBR2+AVG-TH consistently has the worst VMAF (col.1) and PSNR (col.2) scores. This can be attributed to the fact that AVG-TH only relies on past throughput to make bitrate decisions. AERO+Merina⁺

Table 5. Average results of the QoE and its metrics for different network traces for scenario A. ↑: higher is better (green), ↓: lower is better (green), lowest performance (red), and mode: VoD.

		Avg. Perceptual Quality (VMAF)	Avg. Perceptual Quality (PSNR)	Avg. Selected Bitrate (Mbps)	Average RD (%)	Average Duanmu	Average Yu	Average Yin	Avg. QoE ^{itu} (0.46)
5G Amazon	AERO+Pensieve ⁺	63.73	37.93	4.31	0.00 ↓	1.00 ↑	0.98	0.96	3.08
	AERO+Merina ⁺	67.69 ↑	39.48 ↑	3.99	1.13	0.64	0.64	0.13	2.21
	AERO+BBA2-C	66.15	37.81	4.52 ↑	0.00 ↓	1.00 ↑	1.00 ↑	1.00 ↑	3.14 ↑
	BBR2+Pensieve ⁺	55.10	36.27	3.40	0.00 ↓	0.91	0.74	0.59	2.53
	BBR2+Merina ⁺	31.70	33.01	0.63	15.90	0.00	0.14	0.06	1.78
	BBR2+BBA2-C	42.39	34.52	1.87	0.00 ↓	0.72	0.39	0.24	2.04
	Cubic+BBA2-C	63.96	37.47	3.72	0.00 ↓	0.91	0.84	0.90	3.09
	Reno+BBA2-C	59.74	36.84	3.16	0.00 ↓	0.86	0.71	0.76	2.97
	BBR2+AVG-TH	22.82	32.11	0.51	0.00 ↓	0.57	0.12	0.17	1.87
5G Netflix	AERO+Pensieve ⁺	78.02	40.44	8.04 ↑	0.13	1.00 ↑	1.00 ↑	0.42	2.07
	AERO+Merina ⁺	79.03 ↑	41.67 ↑	6.43	0.01	0.97	0.67	0.06	1.79
	AERO+BBA2-C	72.85	39.01	6.94	0.00 ↓	0.94	1.00 ↑	1.00 ↑	3.38 ↑
	BBR2+Pensieve ⁺	44.91	34.56	2.93	0.00 ↓	0.66	0.37	0.15	2.19
	BBR2+Merina ⁺	54.15	36.59	1.66	0.00 ↓	0.55	0.17	0.04	2.05
	BBR2+BBA2-C	65.80	37.74	6.13	0.00 ↓	0.88	0.87	0.79	2.74
	Cubic+BBA2-C	69.98	38.36	5.47	0.00 ↓	0.83	0.79	0.85	3.27
	Reno+BBA2-C	66.21	37.78	4.68	0.00 ↓	0.77	0.66	0.65	3.17
	BBR2+AVG-TH	24.91	32.45	1.06	0.00 ↓	0.48	0.16	0.20	1.88
LTE Belgium	AERO+Pensieve ⁺	77.28	39.97	7.31	0.00 ↓	1.00 ↑	0.64	0.01	1.40
	AERO+Merina ⁺	80.08 ↑	42.03 ↑	6.11	0.00 ↓	0.92	0.46	0.00	1.78
	AERO+BBA2-C	74.90	39.03	8.17 ↑	0.00 ↓	1.00 ↑	1.00 ↑	1.00 ↑	3.53 ↑
	BBR2+Pensieve ⁺	39.75	34.03	1.42	0.00 ↓	0.51	0.14	0.04	2.00
	BBR2+Merina ⁺	36.15	33.79	1.27	6.71	0.29	0.12	0.00	1.81
	BBR2+BBA2-C	46.92	35.10	4.01	0.00 ↓	0.69	0.45	0.25	2.14
	Cubic+BBA2-C	69.10	38.07	6.71	0.00 ↓	0.89	0.80	0.72	3.04
	Reno+BBA2-C	62.38	37.05	4.34	0.00 ↓	0.72	0.48	0.35	2.72
	BBR2+AVG-TH	26.36	32.28	2.33	0.00 ↓	0.59	0.23	0.07	1.90

consistently achieves the highest VMAF and PSNR scores in all traces, while its counterpart BBR2+Merina⁺ lags behind marginally by ~88% in VMAF, far exceeding the JND threshold for VoD as well.

In 5G Amazon trace, although AERO+Merina⁺ causes a slight rebuffering (hinting at further potential improvements on ABR side), it dominates in terms of VMAF and PSNR. AERO+BBA2-C provides the highest average bitrate. In contrast, BBR2+BBA2-C lags behind in terms of VMAF. BBR2+Merina⁺ is the only combination to cause significant rebuffering events.

The results for the 5G Netflix trace align closely with the findings for the 5G Amazon trace with two noticeable differences. AERO+Pensieve⁺ is the combination with the highest average bitrate and while AERO+Merina⁺ provides the lowest QoE^{itu}(0.46-col.8), it is leagues ahead in terms of VMAF, rendering the QoE^{itu}(0.46) as an ineffective metric for overall QoE. The findings for the 5G traces mirror those of the LTE Belgium trace. Overall, the combinations involving AERO for the same ABR demonstrate significant improvements in both VMAF and PSNR scores.

2) Scenario B. Table 6 presents the results for the heterogeneous 4K video (STC) in Scenario B (see Table 2), evaluated across all network traces and combinations of ABR+CC. The observed trends closely resemble those in the VoD mode for Scenario A. AERO +Pensieve⁺ and AERO +Merina⁺ consistently achieve the highest VMAF scores (col. 1), outperforming their respective counterparts by at least 25% across all traces. In addition, they achieve the highest PSNR (col. 2) and average bitrate (col. 3) while maintaining minimal rebuffering (col. 4). These results highlight the significant QoE improvements enabled by the intelligent design of AERO, which dynamically adjusts the congestion window in real time according to prevailing network conditions, ensuring stable and high-quality video streaming performance.

Table 6. Average results of the QoE and its metrics for different network traces for scenario B. ↑: higher is better (green), ↓: lower is better (green), lowest performance (red), and mode: VoD.

		Avg. Perceptual Quality (VMAF)	Avg. Perceptual Quality (PSNR)	Avg. Selected Bitrate (Mbps)	Average RD (%)	Average Duanmu	Average Yu	Average Yin	Avg. QoE ^{itu} (0.46)
5G Amazon	AERO+Pensieve ⁺	81.85	49.05	4.44	0.00 ↓	1.00 ↑	1.00 ↑	1.00 ↑	2.87
	AERO+Merina ⁺	85.61 ↑	50.47 ↑	5.03 ↑	0.14	1.00 ↑	1.00 ↑	1.00 ↑	2.45
	AERO+BBA2-C	81.97	47.67	4.47	0.00 ↓	1.00 ↑	1.00 ↑	1.00 ↑	3.15 ↑
	BBR2+Pensieve ⁺	65.10	44.15	2.02	0.02	0.73	0.42	0.24	1.79
	BBR2+Merina ⁺	62.54	44.39	1.24	0.45	0.40	0.25	0.11	1.91
	BBR2+BBA2-C	59.89	43.07	1.65	0.00 ↓	0.71	0.30	0.05	1.91
	Cubic+BBA2-C	81.46	47.37	4.43	0.00 ↓	1.00 ↑	0.97	0.92	2.94
	Reno+BBA2-C	77.95	46.69	3.34	0.03	0.86	0.73	0.64	2.91
	BBR2+AVG-TH	39.84	39.40	0.51	0.00 ↓	0.70	0.57	1.00 ↑	1.87
5G Netflix	AERO+Pensieve ⁺	90.84 ↑	51.54 ↑	8.72 ↑	0.00 ↓	1.00 ↑	1.00 ↑	1.00 ↑	2.89
	AERO+Merina ⁺	87.17	51.46	6.81	0.22	1.00 ↑	1.00 ↑	1.00 ↑	2.26
	AERO+BBA2-C	85.07	48.54	6.39	0.00 ↓	1.00 ↑	0.98	0.88	2.98
	BBR2+Pensieve ⁺	52.88	41.95	3.02	0.00 ↓	0.64	0.29	0.05	1.95
	BBR2+Merina ⁺	51.68	42.61	1.12	3.90	0.21	0.14	0.01	1.72
	BBR2+BBA2-C	62.47	43.30	2.13	0.07	0.58	0.28	0.08	1.91
	Cubic+BBA2-C	86.42	49.19	6.39	0.00 ↓	1.00 ↑	1.00 ↑	1.00 ↑	3.37 ↑
	Reno+BBA2-C	81.54	47.64	4.41	0.00 ↓	0.83	0.66	0.54	3.05
	BBR2+AVG-TH	42.80	40.16	1.06	0.00 ↓	0.71	0.82	1.00 ↑	1.87
LTE Belgium	AERO+Pensieve ⁺	91.36 ↑	51.15	13.06 ↑	0.08	1.00 ↑	1.00 ↑	1.00 ↑	1.75
	AERO+Merina ⁺	88.78	51.55 ↑	11.29	0.00 ↓	1.00 ↑	1.00 ↑	1.00 ↑	1.62
	AERO+BBA2-C	87.06	48.00	6.95	0.00 ↓	1.00 ↑	1.00 ↑	1.00 ↑	2.67
	BBR2+Pensieve ⁺	66.71	44.37	5.48	0.00 ↓	0.64	0.34	0.12	2.19
	BBR2+Merina ⁺	66.05	44.88	5.79	0.33	0.55	0.47	0.40	2.02
	BBR2+BBA2-C	78.25	45.73	4.25	0.00 ↓	0.79	0.57	0.41	2.28
	Cubic+BBA2-C	86.80	47.99	5.67	0.00 ↓	0.90	0.79	0.74	2.57
	Reno+BBA2-C	82.58	46.89	4.34	0.00 ↓	0.79	0.58	0.46	2.68 ↑
	BBR2+AVG-TH	46.06	40.40	2.88	0.00 ↓	0.96	0.90	0.77	1.90

On average, across all traces and scenarios, AERO improved the VMAF by ~59%, ~88% and ~39%, respectively, in combination with Pensieve⁺, Merina⁺ and BBA2-C against its counterpart BBR2. Our results show that integrating AERO with the same ABR enhances performance and supports better ABR decision-making, improving overall QoE in video streaming. This demonstrates AERO's compatibility with various ABRs for superior streaming outcomes, reinforcing that mean delivery rate is crucial for VoD streaming. The superior QoE achieved by AERO-based configurations stems directly from its transport-layer behavior. As shown in Fig. 4, AERO consistently delivers a higher and more stable throughput compared to BBR2, Cubic, and Reno, while maintaining comparable delay. This stability reduces the short-term variance in estimated bandwidth that ABR algorithms depend on when selecting segment bitrates. For learning-based ABRs such as Pensieve⁺ and Merina⁺, a more predictable throughput allows the policy to confidently select higher-quality representations without triggering stall events or oscillatory quality switches. In contrast, heuristic CC schemes—particularly BBR2 in fluctuating 5G/LTE traces—exhibit abrupt throughput shifts that propagate into overly aggressive or conservative ABR decisions, resulting in either rebuffering (e.g., BBR2+Pensieve⁺) or persistent under-utilization of available bandwidth (e.g., BBR2+BBA2-C). Thus, AERO's impact is not limited to transport efficiency; by smoothing the effective bandwidth signal exposed to the ABR, it enables more stable, higher-quality adaptation behavior, which ultimately drives the observed QoE gains.

4.3.3 QUIC Friendliness and Pluggability of AERO. We extended our architecture from Fig.3 to support 5 clients, where a single server is connected to all clients with a one-to-many mapping, with all clients simultaneously requesting the video content. This modification allows us to evaluate the QUIC friendliness of AERO

Table 7. Average results over 5 clients of the QoE and its metrics for 5G Amazon trace for scenario A for different QUIC implementations. $\uparrow$: higher is better (green), $\downarrow$: lower is better (green), lowest performance (red), and mode: VoD.

		Avg. Perceptual Quality (VMAF)	Avg. Perceptual Quality (PSNR)	Avg. Selected Bitrate (Mbps)	Average RD (%)	Average Duanmu	Average Yu	Average Yin	Avg. QoE ^{itu} (0.46)
Quinn	AERO+Pensieve ⁺	72.75	39.63	6.77	0.00 $\downarrow$	0.94	0.88	0.74	3.08
	AERO+BBA2-C	71.47	38.85	5.92	0.00 $\downarrow$	0.92	0.87	0.84	3.33 $\uparrow$
	BBR+Pensieve ⁺	73.52 $\uparrow$	39.90 $\uparrow$	6.97 $\uparrow$	0.00 $\downarrow$	0.93	0.84	0.81	3.26
	BBR+BBA2-C	69.85	38.47	5.29	0.00 $\downarrow$	0.96	0.91	0.91	3.27
	Cubic+BBA2-C	66.23	37.78	4.50	0.00 $\downarrow$	0.96	0.90	0.86	3.20
	Reno+BBA2-C	66.16	37.77	4.40	0.00 $\downarrow$	0.98 $\uparrow$	0.93 $\uparrow$	0.92 $\uparrow$	3.20
Aioquic	AERO+Pensieve ⁺	66.98 $\uparrow$	38.41 $\uparrow$	4.48 $\uparrow$	0.00 $\downarrow$	0.92	0.94	0.83	2.76
	AERO+BBA2-C	66.16	37.74	4.46	0.00 $\downarrow$	0.97	0.95 $\uparrow$	0.94 $\uparrow$	3.19 $\uparrow$
	Cubic+Pensieve ⁺	62.91	37.57	3.44	0.00 $\downarrow$	0.98 $\uparrow$	0.94	0.77	2.38
	Cubic+BBA2-C	58.87	36.53	3.47	0.00 $\downarrow$	0.98 $\uparrow$	0.92	0.90	2.77
	Reno+Pensieve ⁺	65.65	38.03	3.89	0.00 $\downarrow$	0.95	0.91	0.70	2.66
	Reno+BBA2-C	61.15	36.98	3.63	0.00 $\downarrow$	0.98 $\uparrow$	0.94	0.90	2.94

in comparison with other CC and to observe how multiple QUIC flows compete with each other. While we initially aimed to extend Quiche for this purpose, its lack of support for a multi-client setup due to the absence of an asynchronous or multi-threaded API for data transmission proved a limitation. However, since the metrics required for the AERO architecture are readily accessible across various QUIC implementations (usually in the congestion controller class), integrating AERO into different QUIC stacks is straightforward. As a result, we successfully incorporated AERO into other QUIC implementations, specifically Quinn and Aioquic, highlighting the pluggability and ease of integration of AERO. Although PBQ-enhanced QUIC [62] is also a DRL-based architecture, the code/GitHub for that is not publicly available. Still, PBQ is built on top of BBR, and many QUIC implementations like Aioquic and Kwik do not support it by default. Table 7 illustrates the impact of QUIC CC on ABR for multiple QUIC implementations. The results presented are the average over 5 clients. Note that Quinn only supports BBR, Cubic, and Reno while Aioquic supports only Cubic and Reno. We observed scenario-wide similar results for all network traces. Due to space constraints, the results for only one scenario and network trace (5G Amazon) are shown.

For Quinn, BBR+Pensieve⁺ is the combination with the highest VMAF, with AERO+Pensieve⁺ trailing slightly behind. For Aioquic, AERO+Pensieve⁺ achieves the highest VMAF, PSNR and average bitrate. The ABR combinations with Cubic and Reno have similar results.

To visualize how multiple QUIC flows compete over time, we provide per-CC time-series plots of the delivery rate for all five clients in Appendix G. While these time-series confirm that all clients remain active throughout the session, the primary quantitative evidence for fairness is presented in Fig. 6, which demonstrates that AERO maintains fairness among concurrent QUIC flows: all clients achieve nearly identical average video bitrates, and the resulting Jain's Fairness Index is 0.995 (on a 0–1 scale, where 1 indicates perfectly equal allocation), confirming near-perfect equitable bandwidth distribution across clients.

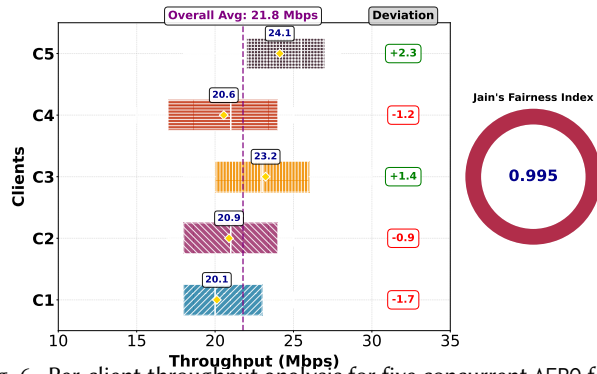


Fig. 6. Per-client throughput analysis for five concurrent AERO flows using the Quinn implementation on the 5G Amazon trace. The Jain's Fairness Index of 0.995 indicates equitable bandwidth distribution.

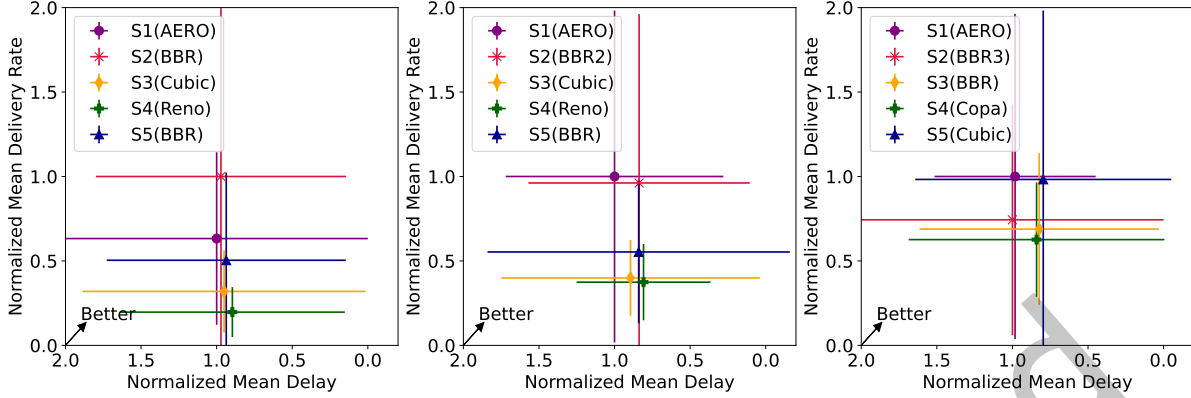


Fig. 7. QUIC competitiveness for Quinn (left), Quiche (center) and Tquic (right) for LTE Belgium trace.

4.3.4 QUIC Competitiveness for different CC. We further extended our setup from Fig.3 to support multiple servers serving multiple clients running different CCs connected to the same shaper as link bottleneck. We connected 5 servers to 5 clients with a one-to-one mapping over a single shaper. Furthermore, we integrated AERO with Tquic as well. As shown in Fig.7, for Quinn, all CC algorithms exhibit comparable delay, while BBR achieves the highest delivery rate with AERO as the runner-up. For quiche, both AERO and BBR provide the highest delivery rate, with AERO having a slightly higher delay compared to other algorithms. For Tquic, AERO and Cubic achieve the highest delivery rates, with the remaining algorithms as runner-ups, but AERO and BBR3 experience more delay. These observations suggest that AERO tends to be slightly aggressive when competing with other CC for the same bottleneck capacity, indicating the potential need for re-training AERO with an awareness of other QUIC flows, which we earmark for future exploration.

5 Discussion and Limitations

AERO's primary strength is its pluggable, implementation-agnostic design, which enables seamless integration into any QUIC stack without kernel-level modifications. This ensures compatibility across diverse implementations—such as Quiche, Aioquic, and Tquic—and, by operating entirely in user space, aligns with QUIC's architectural principles while facilitating rapid experimentation and debugging. AERO also delivers consistent gains in both transport-level efficiency and application-layer QoE across varying network conditions and streaming modes, while its single-agent formulation remains computationally lightweight. However, this single-agent design introduces limitations in multi-client or shared-bottleneck settings, where independent agents compete without explicit coordination. This is visible in Section 4.3.4, where AERO shows stronger bandwidth acquisition when coexisting with other CC algorithms at the same bottleneck—yielding high throughput efficiency but reduced fairness with heterogeneous traffic. This reflects the absence of shared-bottleneck or fairness-aware objectives during training rather than instability, making coexistence with diverse QUIC and TCP flows an important practical consideration.

To address these issues, future work will extend AERO toward a multi-agent reinforcement learning (MARL) framework for multi-path QUIC (MP-QUIC) and multi-user scenarios. To address coexistence with heterogeneous (non-AERO) flows, such a MARL framework could incorporate an overload penalty term in the reward function and employ a centralized critic with access to aggregate bottleneck statistics, enabling AERO agents to learn bandwidth-sharing policies that remain fair even when competing against flows governed by different CC algorithms. In this context, shared bottleneck detection mechanisms such as MPQUIC-SBD [30] would be particularly relevant for coordinating AERO's congestion responses across multiple QUIC subflows sharing common network resources. Other promising directions include cross-layer coordination between AERO's transport decisions and client-side

ABR selection to jointly optimize throughput, latency, and perceptual quality, and online adaptation mechanisms that update the policy on-the-fly in response to evolving network conditions.

6 Conclusion

This paper presented AERO, the first learning-based, pluggable, and implementation-agnostic congestion control algorithm for QUIC. Unlike existing heuristic or kernel-coupled DRL approaches, AERO offers a modular user-space design that can be integrated into any QUIC implementation with minimal engineering effort. Leveraging an A2C-PPO architecture with Generalized Advantage Estimation, AERO dynamically adjusts the congestion window based on real-time network feedback, effectively adapting to fluctuating link conditions. Comprehensive evaluations across multiple network traces, QUIC stacks, and adaptive bitrate algorithms demonstrated that AERO consistently outperforms state-of-the-art congestion control schemes—achieving up to 13% higher perceptual video quality (VMAF) and up to 30% lower rebuffering in VoD, as well as 12% higher VMAF and 65% fewer stalls in live low-latency streaming scenarios. Beyond its empirical success, AERO establishes a generalizable foundation for learning-driven congestion control in QUIC. Its design principles—pluggability, modularity, and adaptability—make it a viable blueprint for integrating machine learning into transport-layer decision-making. Future extensions of AERO toward multi-path, multi-agent, and cross-layer learning promise to further bridge the gap between transport intelligence and multimedia quality optimization, ultimately contributing to a new generation of self-optimizing QUIC networks.

References

- [1] 2023. THE GLOBAL INTERNET PHENOMENA REPORT JANUARY 2023. <https://www.sandvine.com/phenomena>.
- [2] Soheil Abbasloo, Chen-Yu Yen, and H Jonathan Chao. 2020. Classic meets modern: A pragmatic learning-based congestion control for the internet. In *ACM SIGCOMM*.
- [3] Rishabh Agarwal, Max Schwarzer, Pablo Samuel Castro, Aaron C Courville, and Marc Bellemare. 2021. Deep reinforcement learning at the edge of the statistical precipice. *Advances in neural information processing systems* 34 (2021), 29304–29320.
- [4] Karl Johan Åström and Richard Murray. 2021. *Feedback systems: an introduction for scientists and engineers*. Princeton university press.
- [5] Ali C Begen, Mehmet N Akcay, Abdelhak Bentaleb, and Alex Giladi. 2022. Adaptive streaming of content-aware-encoded videos in dash.js. *SMPTe Motion Imaging Journal* 131, 4 (2022), 30–38.
- [6] Abdelhak Bentaleb, Ali C Begen, Saad Harous, and Roger Zimmermann. 2020. Data-driven bandwidth prediction models and automated model selection for low latency. *IEEE Transactions on Multimedia* 23 (2020), 2588–2601.
- [7] Abdelhak Bentaleb, May Lim, Mehmet N. Akcay, Ali C. Begen, and Roger Zimmermann. 2024. Bitrate Adaptation and Guidance with Meta Reinforcement Learning. *IEEE Transactions on Mobile Computing* (2024). To appear.
- [8] Abdelhak Bentaleb, Bayan Taani, Ali C Begen, Christian Timmerer, and Roger Zimmermann. 2018. A survey on bitrate adaptation schemes for streaming media over HTTP. *IEEE Communications Surveys & Tutorials* 21, 1 (2018), 562–585.
- [9] c3m3gyanesh. 2023. Working with Longer duration video. https://github.com/Telecommunication-Telemedia-Assessment/bitstream_mode3_p1204_3/issues/31. Accessed on Oct. 20, 2024.
- [10] Sheng Chen, Y. Li, and X. Zhang. 2024. *Delivery Rate Estimation for Congestion Control*. Technical Report draft-cheng-iccr-g-delivery-rate-estimation-01. IETF. <https://www.ietf.org/archive/id/draft-cheng-iccr-g-delivery-rate-estimation-01.html> Work in Progress.
- [11] Zhenzhong Chen and Hongyi Liu. 2014. JND modeling: Approaches and applications. In *2014 19th International Conference on Digital Signal Processing*. 827–830. <https://doi.org/10.1109/ICDSP.2014.6900782>
- [12] Quentin De Coninck, François Michel, Maxime Piroux, Florentin Rochet, Thomas Given-Wilson, Axel Legay, Olivier Pereira, and Olivier Bonaventure. 2019. Pluginizing QUIC. In *ACM SIGCOMM*. 59–74.
- [13] Zhengfang Duanmu, Abdul Rehman, and Zhou Wang. 2018. A quality-of-experience database for adaptive video streaming. *IEEE Transactions on Broadcasting* 64, 2 (2018), 474–487.
- [14] Logan Engstrom, Andrew Ilyas, Shibani Santurkar, Dimitris Tsipras, Firdaus Janoos, Larry Rudolph, and Aleksander Madry. 2019. Implementation matters in deep rl: A case study on ppo and trpo. In *International conference on learning representations*.
- [15] Frédéric Garcia and Emmanuel Rachelson. 2013. Markov decision processes. *Markov Decision Processes in Artificial Intelligence* (2013), 1–38.
- [16] QUIC Working Group. 2024. *QUIC Implementations*. <https://github.com/quicwg/base-drafts/wiki/Implementations> Accessed: 2024-07-21.
- [17] Zafer Gurel, Tugce Erkilic Civelek, Atakan Bodur, Senem Bilgin, Deniz Yeniceri, and Ali C Begen. 2023. Media over QUIC: Initial testing, findings and results. In *ACM MM Sys*.

- [18] Stephen Hemminger et al. 2005. Network emulation with NetEm. In *Linux conf au*, Vol. 5. 2005.
- [19] Peter Henderson, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. 2018. Deep reinforcement learning that matters. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 32.
- [20] Joris Herbots, Mike Vandersanden, Peter Quax, and Wim Lamotte. 2023. Vegvisir: A testing framework for HTTP/3 media streaming. In *ACM MMSys*.
- [21] Joris Herbots, Arno Verstraete, Maarten Wijnants, Peter Quax, and Wim Lamotte. 2023. Cross that boundary: Investigating the feasibility of cross-layer information sharing for enhancing ABR decision logic over QUIC. In *Proceedings of the 33rd Workshop on Network and Operating System Support for Digital Audio and Video*. 50–57.
- [22] Tianchi Huang, Chao Zhou, Rui-Xiao Zhang, Chenglei Wu, Xin Yao, and Lifeng Sun. 2019. Comyco: Quality-aware adaptive video streaming via imitation learning. In *ACM MM*.
- [23] Te-Yuan Huang, Ramesh Johari, Nick McKeown, Matthew Trunnell, and Mark Watson. 2014. A buffer-based approach to rate adaptation: Evidence from a large video streaming service. In *ACM SIGCOMM*.
- [24] ITU-T. 2024. ITU-T P.1203: Subjective video quality assessment methods for multimedia applications. <https://github.com/itu-p1203/itu-p1203>.
- [25] J Iyengar and I Swett. 2021. Rfc 9002: Quic loss detection and congestion control.
- [26] J. Iyengar and I. Swett. 2021. RFC 9002: QUIC Loss Detection and Congestion Control and Stream. RFC 9002. IETF. <https://www.rfc-editor.org/rfc/rfc9002.html>
- [27] Van Jacobson. 1988. Congestion avoidance and control. *ACM SIGCOMM computer communication review* 18, 4 (1988), 314–329.
- [28] Nathan Jay, Noga Rotman, Brighten Godfrey, Michael Schapira, and Aviv Tamar. 2019. A deep reinforcement learning perspective on internet congestion control. In *International Conference on Machine Learning*. PMLR, 3050–3059.
- [29] Nuowen Kan, Yuankun Jiang, Chenglin Li, Wenrui Dai, Junni Zou, and Hongkai Xiong. 2022. Improving generalization for neural adaptive video streaming via meta reinforcement learning. In *ACM MM*.
- [30] Bruno Yuji Lino Kimura, Simone Ferlin, Thomas William do Prado Paiva, Toktam Mahmoodi, Anna Brunström, and Ozgu Alay. 2025. Evaluating Adaptive Video Streaming over Multipath QUIC with Shared Bottleneck Detection. *ACM Transactions on Multimedia Computing, Communications, and Applications* 21, 9 (2025), 1–25.
- [31] Adam Langley, Alistair Riddoch, Alyssa Wilk, et al. 2017. The QUIC Transport Protocol: Design and Internet-Scale Deployment. In *Proceedings of SIGCOMM (Los Angeles, CA, USA) (SIGCOMM '17)*. Association for Computing Machinery, New York, NY, USA, 183–196. <https://doi.org/10.1145/3098822.3098842>
- [32] Zhi Li, Christos Bampis, Julie Novak, Anne Aaron, Kyle Swanson, Anush Moorthy, and JD Cock. 2018. VMAF: The journey continues. *Netflix Technology Blog* 25, 1 (2018).
- [33] Zhi Li, Xiaoqing Zhu, Joshua Gahm, Rong Pan, Hao Hu, Ali C Begen, and David Oran. 2014. Probe and adapt: Rate adaptation for HTTP video streaming at scale. *IEEE journal on selected areas in communications* 32, 4 (2014), 719–733.
- [34] Xudong Liao, Han Tian, Chaoliang Zeng, Xinchun Wan, and Kai Chen. 2024. Astraea: Towards Fair and Efficient Learning-based Congestion Control. In *EuroSys*. 99–114.
- [35] Jinting Lin, Lin Cui, Yuxiang Zhang, Fung Po Tso, and Quanlong Guan. 2019. Extensive evaluation on the performance and behaviour of TCP congestion control protocols under varied network scenarios. *Computer Networks* 163 (2019), 106872.
- [36] Josip Lorincz, Zvonimir Klarin, and Julije Ožegović. 2021. A comprehensive overview of TCP congestion control in 5G networks: Research challenges and future perspectives. *Sensors* 21, 13 (2021), 4510.
- [37] Hongzi Mao, Ravi Netravali, and Mohammad Alizadeh. 2017. Neural adaptive video streaming with pensieve. In *ACM SIGCOMM*.
- [38] Robin Marx, Wim Lamotte, Jonas Reynders, Kevin Pittevels, and Peter Quax. 2018. Towards QUIC debuggability. In *Proceedings of the Workshop on the Evolution, Performance, and Interoperability of QUIC*. 1–7.
- [39] Microsoft. 2023. *Bandwidth Estimation Challenge*. <https://www.microsoft.com/en-us/research/academic-program/bandwidth-estimation-challenge/>. Accessed: 2023.
- [40] Lorenzo Pappone, Alessio Sacco, and Flavio Esposito. 2025. Mutant: Learning Congestion Control from Existing Protocols via Online Reinforcement Learning. In *USENIX NSDI*. 1507–1522.
- [41] quic-go. 2024. quic-go: An implementation of the QUIC protocol in Go. <https://github.com/quic-go/quic-go>.
- [42] Darijo Raca, Dylan Leahy, Cormac J Sreenan, and Jason J Quinlan. 2020. Beyond throughput, the next generation: A 5G dataset with channel and context metrics. In *ACM MM*.
- [43] Darijo Raca, Maëlle Manificier, and Jason J Quinlan. 2020. Godash—go accelerated has framework for rapid prototyping. In *IEEE QoMEX*.
- [44] David Schinazi, Fan Yang, and Ian Swett. 2020. Chrome is deploying HTTP/3 and IETF QUIC. Retrieved October 8 (2020), 2020.
- [45] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347* (2017).
- [46] Jashanjot Singh Sidhu and Abdelhak Bentaleb. 2025. Aero: A Pluggable Congestion Control for QUIC. In *Proceedings of the 35th Workshop on Network and Operating System Support for Digital Audio and Video*. 29–35.

- [47] Jashanjot Singh Sidhu and Abdelhak Bentaleb. 2026. Video streaming over QUIC: A comprehensive study. *ACM Transactions on Multimedia Computing, Communications and Applications* 22, 4 (2026), 1–29.
- [48] slhck. 2020. Memory issues. https://github.com/Telecommunication-Telemedia-Assessment/bitstream_mode3_p1204_3/issues/16.
- [49] Kevin Spiteri, Rahul Uргаonkar, and Ramesh K Sitaraman. 2020. BOLA: Near-optimal bitrate adaptation for online videos. *IEEE/ACM transactions on networking* 28, 4 (2020), 1698–1711.
- [50] Richard S Sutton, Andrew G Barto, et al. 1998. *Reinforcement learning: An introduction*. Vol. 1. MIT press Cambridge.
- [51] Alexander Tanchenko. 2014. Visual-PSNR measure of image quality. *Journal of Visual Communication and Image Representation* 25, 5 (2014), 874–878.
- [52] Babak Taraghi, Hadi Amirpour, and Christian Timmerer. 2022. Multi-codec ultra high definition 8K MPEG-DASH dataset. In *ACM MM*.
- [53] Han Tian, Xudong Liao, Decang Sun, Chaoliang Zeng, Yong Wang, and Kai Chen. 2025. Achieving Fairness Generalizability for Learning-based Congestion Control with Jury. In *EuroSys*. 413–427.
- [54] Jeroen Van Der Hooft, Stefano Petrangeli, Tim Wauters, Rafael Huysegems, Patrice Rondao Alface, Tom Bostoen, and Filip De Turck. 2016. HTTP/2-based adaptive streaming of HEVC video over 4G/LTE networks. *IEEE Communications Letters* 20, 11 (2016), 2177–2180.
- [55] Keith Winstein and Hari Balakrishnan. 2013. Tcp ex machina: Computer-generated congestion control. *ACM SIGCOMM Computer Communication Review* 43, 4 (2013), 123–134.
- [56] Francis Y Yan, Hudson Ayers, Chenzhi Zhu, Sadjad Fouladi, James Hong, Keyi Zhang, Philip Levis, and Keith Winstein. 2020. Learning in situ: a randomized experiment in video streaming. In *USENIX NSDI*.
- [57] Yang Richard Yang, Min Sik Kim, Xincheng Zhang, and Simon S Lam. 2000. Two problems of tcp aimd congestion control. *Department of Computer Sciences, UT Austin, Tech. Rep. TR-00-13* (2000).
- [58] Chen-Yu Yen, Soheil Abbasloo, and H Jonathan Chao. 2023. Computers Can Learn from the Heuristic Designs and Master Internet Congestion Control. In *ACM SIGCOMM*.
- [59] Xiaoqi Yin, Abhishek Jindal, Vyas Sekar, and Bruno Sinopoli. 2015. A control-theoretic approach for dynamic adaptive video streaming over HTTP. In *ACM SIGCOMM*.
- [60] Li Yu, Tammam Tillo, and Jimin Xiao. 2017. QoE-driven dynamic adaptive video streaming strategy with future information. *IEEE Transactions on Broadcasting* 63, 3 (2017), 523–534.
- [61] Xuan Zeng, Haoran Xu, Chen Chen, Xumiao Zhang, et al. 2025. Learning Production-Optimized Congestion Control Selection for Alibaba Cloud CDN. In *USENIX NSDI*. 823–840.
- [62] Zhifei Zhang, Shuo Li, Yiyang Ge, Ge Xiong, Yu Zhang, and Ke Xiong. 2023. PBQ-Enhanced QUIC: QUIC with Deep Reinforcement Learning Congestion Control Mechanism. *Entropy* 25, 2 (2023), 294.

Online Appendix

A Ethics

In accordance with the guidelines set by the Human Ethics Research Committee (HERC) of our university, our research adheres to all ethical standards. All implementations utilized in this study are open-source and publicly available, ensuring transparency and avoiding any ethical concerns. Furthermore, our analysis was conducted without any attempt to link video streaming sessions to personal identities, maintaining strict compliance with ethical research practices.

B Notation List

Table 8 outlines the symbols utilized across the entire paper.

Table 8. AERO notation table.

Symbol	Description	Symbol	Description
<i>General</i>		<i>Reward & cwnd update</i>	
MI	Monitor interval	th	Throughput
t	Time step	d	Delay
$s, \vec{s}$	State, state vector	l	Packet loss ratio
r	Reward	j	Jitter
k	Total no. of timesteps	w_1, w_2, w_3	Delay, loss, jitter penalties
cwnd	Congestion window	$cwnd$	Predicted cwnd
A	Base factor for cwnd	$E[cwnd]$	Expected cwnd
α	Power factor for A	$cwnd_{max}$	Maximum cwnd
<i>MDP</i>		ϕ	cwnd difference threshold
$\mathcal{S}, \mathcal{A}$	State, action space	ρ	Extra gain/penalty factor
$\mathcal{P}$	State transition prob. function	<i>Policy gradient & losses</i>	
$\mathcal{R}$	Expected immediate rewards	T_{π_0}	Policy gradient batch size
π	DRL policy	γ	Discount factor
Λ	Accum. discounted advantage	δ	TD error
<i>Hyperparameters</i>		V	Value function
$\omega, \hat{\omega}$	Actor, critic learning rate	λ	Advantage estimator
ϵ	Clip parameter	L^{clip}	Objective function loss
δ	Entropy coefficient	$\mathfrak{R}$	New/old policy prob. ratio
T_{θ}, n_s	PPO update interval, steps	L^{value}	Value function loss
B_s	Batch size	G	Return
θ	Exploration size	L^{ent}, L^{total}	Entropy, total policy loss
n_e	No. of episodes	∇	Gradient
itr	No. of epochs		

C Network Traces

Figure 8 illustrates the throughput variations across all network traces utilized for AERO training, namely Lumous, LTE Belgium, Twitch and Cascade with average bandwidths (and sampling interval) of 2.4 Mbps (2s), 3.3 Mbps (1s), 1.4 Mbps (5s) and 0.82 Mbps (30s), respectively.

D AERO Hyperparameter Tuning

We executed the model over 50 epochs using a strategy akin to grid search and used bayesian optimization techniques to carry out the hyperparameter tuning for AERO model. Fig. 9 illustrates

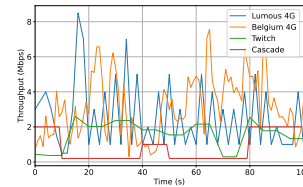


Fig. 8. Bandwidth fluctuations for all network traces.

the significance of hyperparameters identified through our experimentation process during the training phase. We focused our efforts towards fine-tuning parameters that were identified as having the highest importance like exploration size, PPO policy update steps and batch size.

Fig. 10 shows the normalized mean delivery rate vs normalized mean delay results for different AERO models. The traces used for AERO training evaluation are presented in Figure 8. AERO denotes our final fine-tuned NN model, AERO⁺ denotes the NN model with a high base factor of 4, AERO⁻ denotes the NN model with a low base factor of 1 and $\overline{\text{AERO}}$ denotes the NN model without tuning. AERO⁺ lags behind in terms of delivery rate and performs comparably on delay but we noticed that it causes the ABR to completely stall 2/3 times by sending more data when the link capacity is low, highlighting the risk of a drastic update to congestion window and upon congestion detection event, responds by considerably reducing the sending rate as well, leading to its lower delivery rate than AERO. AERO⁻ closely resembles AERO in terms of delay on average but considerably lags behind in providing the optimal delivery rate. Apart from Cascade, $\overline{\text{AERO}}$ is the worst performing scheme on both fronts, underscoring the imperative need for hyperparameter tuning in any DRL architecture. AERO with the fine-tuned model is consistently the best performing scheme across all traces.

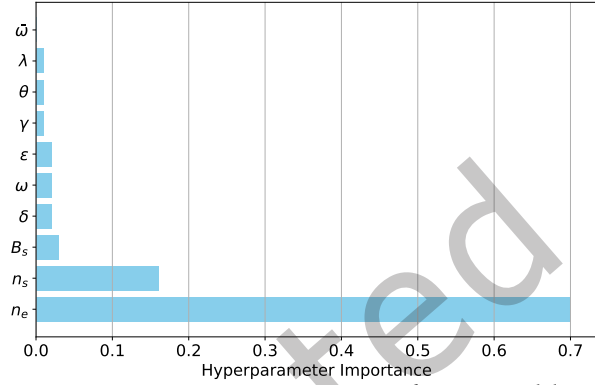


Fig. 9. Hyperparameter Importance for AERO model.

E AERO Computational Assessment

We assessed the computational load imposed by the AERO model on the system. During training, AERO requires only 0.25 s per epoch with 20.18 MB of GPU memory and 688,864 FLOPS. During testing, inference takes 5.43×10^{-4} s per decision, with CPU usage of 22.36 MB and the same 688,864 FLOPS. These figures confirm that AERO is computationally lightweight in both training and inference, making it well-suited for diverse applications and resource-constrained environments.

F Violin Plots for CC Evaluation

We provide violin plots that illustrate the full distribution of normalized mean delivery rate and normalized mean delay across experimental runs for each CC algorithm and network trace.

Fig. 11 and Fig. 12 present the distributions for the LLL streaming mode, while Fig. 13 and Fig. 14 present the corresponding distributions for VoD. In each violin, the solid black line indicates the mean, the dashed red line indicates the median, and the shape of the violin reflects the probability density of the underlying data. These plots confirm the observations from the main evaluation: AERO consistently achieves the highest delivery rate with a tight distribution, indicating both superior performance and low variability across runs. In contrast, BBR2 exhibits a wide spread in delivery rate, reflecting inconsistent link utilization. Cubic and Reno remain concentrated at low delivery rates across all traces and modes. For delay, AERO maintains comparable or only slightly higher delay than competing algorithms, with a consistently narrow distribution, further demonstrating its stability.

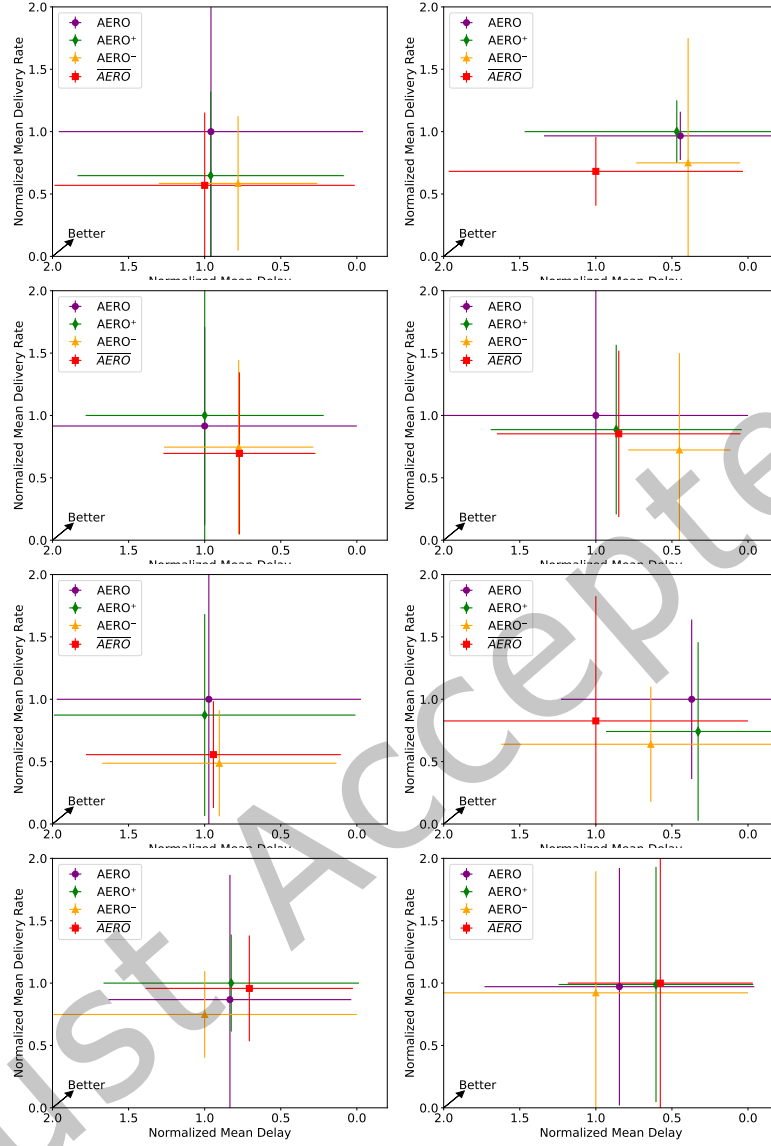


Fig. 10. AERO model fine-tuning in LLL (left) and VoD (right) for Lumous, Belgium, Twitch and Cascade network traces (top to bottom).

G Per-Client Delivery Rate Time Series

Fig. 15 presents the per-client delivery rate time series for five concurrent clients sharing the same bottleneck link over the 5G Amazon trace for the Quinn implementation. Each subplot corresponds to a different CC algorithm: AERO, BBR, Cubic, and Reno. While the overlapping traces make fine-grained comparison challenging, these plots confirm that all five clients remain active and receive bandwidth throughout the streaming session under

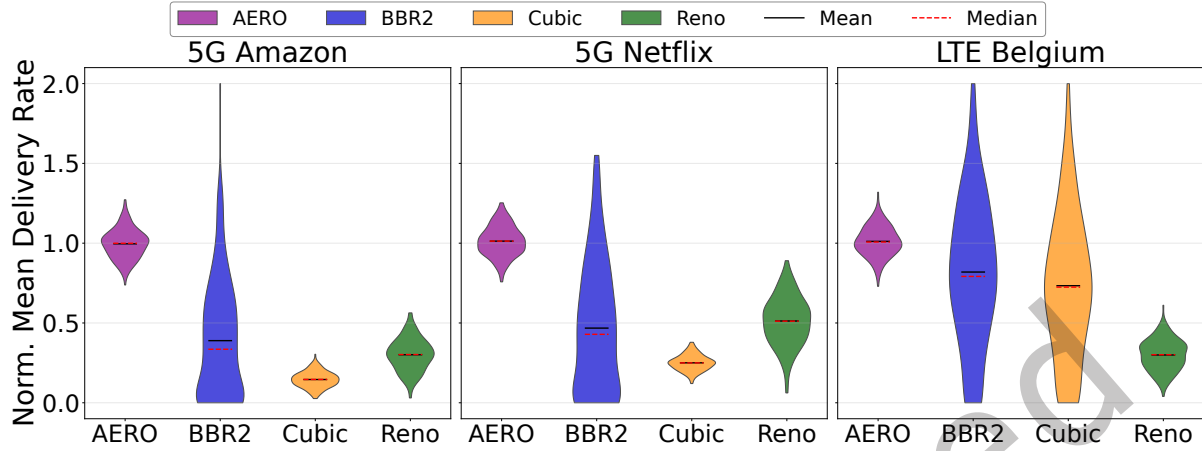


Fig. 11. Violin plot of normalized mean delivery rate in LLL mode for 5G Amazon (left), 5G Netflix (center), and LTE Belgium (right).

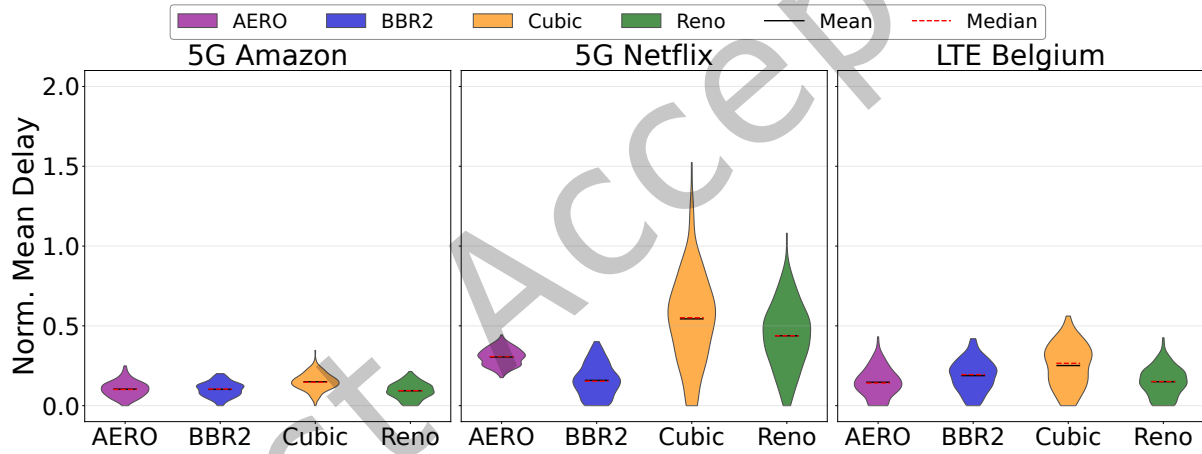


Fig. 12. Violin plot of normalized mean delay in LLL mode for 5G Amazon (left), 5G Netflix (center), and LTE Belgium (right).

each CC. The quantitative fairness analysis, including per-client average throughput and Jain's Fairness Index, is presented in Fig. 6 in the main text.

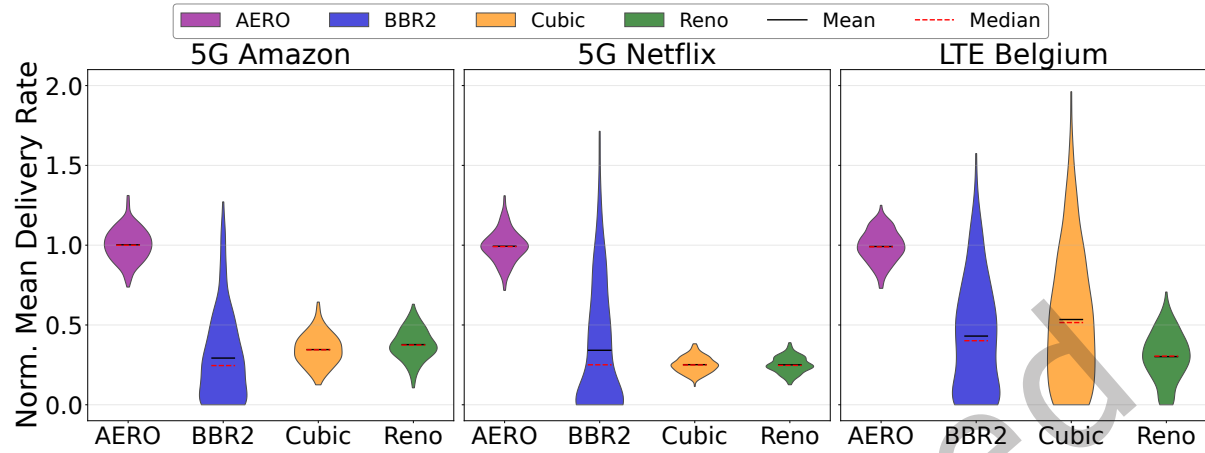


Fig. 13. Violin plot of normalized mean delivery rate in VoD mode for 5G Amazon (left), 5G Netflix (center), and LTE Belgium (right).

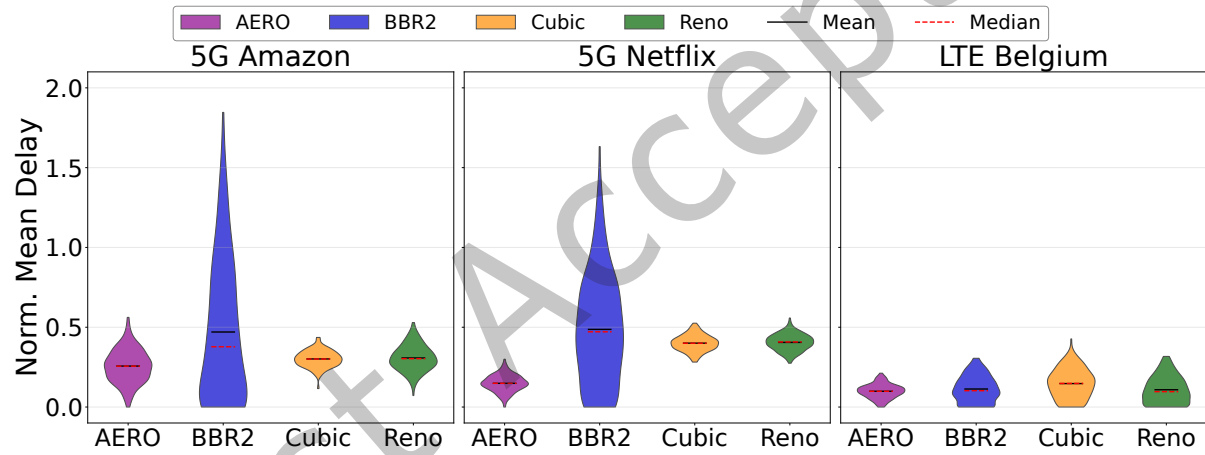


Fig. 14. Violin plot of normalized mean delay in VoD mode for 5G Amazon (left), 5G Netflix (center), and LTE Belgium (right).

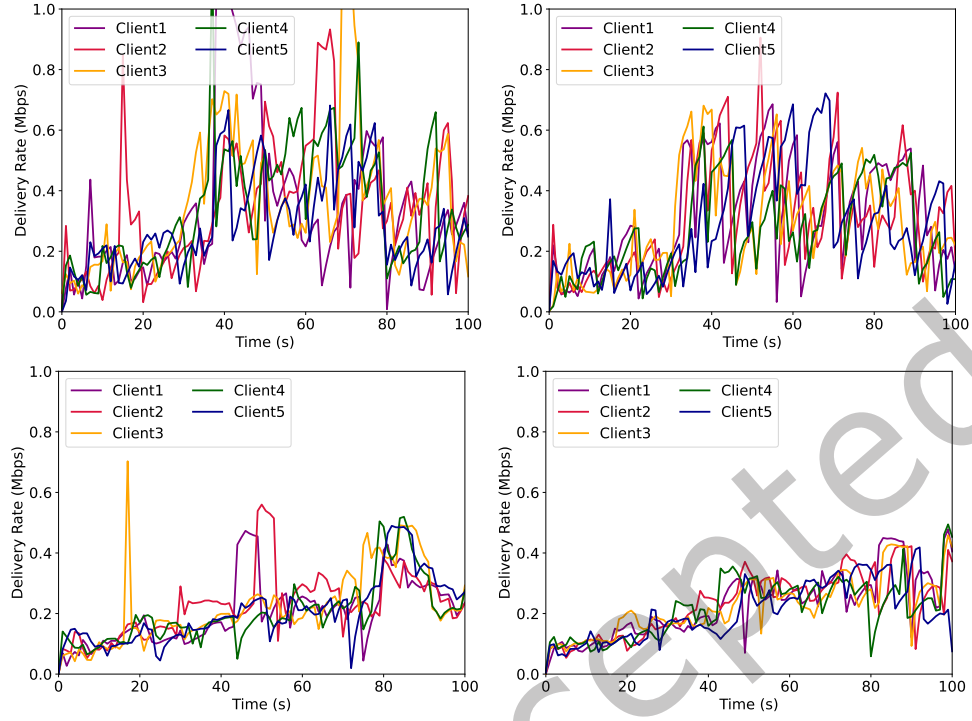


Fig. 15. QUIC friendliness of Quinn for AERO, BBR, Cubic and Reno (left to right) with 5 clients over 5G Amazon trace.